


# KTO KARATAY ÜNİVERSİTESİ

Mühendislik ve Doğa Bilimleri Fakültesi

Bilgisayar Mühendisliği Bölümü

## PYTHON İLE UYGULAMA GELİŞTİRME

### DÖNEM SONU PROJE RAPORU

# Video Oyunu Hit Tahmincisi

*DNN + XGBoost Ensemble Modeli ile Satış Başarısı Tahmini*

**Mustafa ARIKAN**

**221450043**

07.05.2026

# İÇİNDEKİLER

**ÖZET**

**1. GİRİŞ VE PROBLEM TANIMI**

- 1.1. Problem Tanımı ve Motivasyon
- 1.2. Proje Hedefleri
- 1.3. Proje Yapısı ve Pipeline Mimarisi

**2. VERİ SETİ**

- 2.1. Veri Seti Tanıtımı ve Kaynağı
- 2.2. Veri Seti Sütunlarının Ayrıntılı İncelemesi
- 2.3. Veri Temizleme ve Ön İşleme
- 2.4. Özellik Mühendisliği (Feature Engineering)

**3. TEMEL PYTHON PROGRAMLAMA**

- 3.1. Fonksiyonlar ve Modüler Tasarım
- 3.2. Dosya İşlemleri
- 3.3. Hata Yönetimi (try-except)
- 3.4. Veri Yapıları: Lists ve Dicts

**4. NESNE YÖNELİMLİ PROGRAMLAMA (OOP)**

- 4.1. DataLoader Sınıfı
- 4.2. DataAnalyzer Sınıfı
- 4.3. Visualizer Sınıfı
- 4.4. ModelTrainer Sınıfı

**5. NUMPY İLE SAYISAL ANALİZ**

- 5.1. NumPy'ye Dönüşüm ve Betimsel İstatistikler
- 5.2. Pearson Korelasyon Analizi
- 5.3. Polinom Regresyon (np.polyfit)

**6. PANDAS İLE VERİ ANALİZİ**

- 6.1. Veri Okuma ve İlk İnceleme
- 6.2. GroupBy Analizleri

**7. MATPLOTLİB İLE GÖRSELLEŞTİRME**

- 7.1–7.9. Grafik Açıklamaları ve Yorumları

**8. MODEL GELİŞTİRME (TENSORFLOW / XGBOOST)**

- 8.1. Model Mimarisi: DNN
- 8.2. Model Mimarisi: XGBoost
- 8.3. Ensemble Yaklaşımı
- 8.4. Eğitim Teknikleri ve Önlemler
- 8.5. Eşik Optimizasyonu
- 8.6. Deneysel Sonuçlar ve Değerlendirme

**9. GENEL DEĞERLENDİRME VE SONUÇ**

**KAYNAKÇA**

**EKLER — TAM KAYNAK KODLAR**

# ÖZET

Bu proje, video oyunu satış verisini kullanarak bir oyunun ticari anlamda başarılı olup olmayacağını — yani 1 milyon üzerinde küresel satış rakamına ulaşıp ulaşmayacağını — tahmin eden uçtan uca (end-to-end) bir makine öğrenmesi sistemi geliştirmeyi amaçlamaktadır.

Kaggle platformundan temin edilen, 1975–2016 yılları arasında piyasaya çıkan 16.719 video oyununun platformu, türü, yayıncısı, yayın yılı, bölgesel satışları ve eleştirmen/kullanıcı puan bilgilerini içeren gerçek bir veri seti kullanılmıştır. Veri seti; yoğun eksik veri (~%51 oranında puansız kayıt), ciddi sınıf dengesizliği (yalnızca %12 oranında hit oyun) ve kategorik özelliklerin model için bilgilendirici biçimde temsil edilemediği label encoding problemi gibi gerçek dünya veri bilimi zorluklarına sahiptir.

Bu zorluklara çözüm olarak; eksik puan verileri için sentinel değer (-1) ve ikili bayrak özelliği yaklaşımı, kategorik özellikler için target encoding yöntemi ve sınıf dengesizliği için SMOTE (Synthetic Minority Over-sampling Technique) aşırı-örnekleme tekniği kullanılmıştır. Model tarafında, TensorFlow/Keras ile geliştirilen 4 katmanlı Derin Sinir Ağı (DNN) ile XGBoost gradyan artırma sınıflandırıcısı bir ensemble (topluluk öğrenmesi) yapısında birleştirilerek her birinin güçlü yönlerinden yararlanılmıştır.

Test seti üzerinde elde edilen nihai sonuçlar: Doğruluk %88,41, AUC-ROC %89,21, F1 Skoru %57,19 (Precision %52,62, Recall %62,62). Tüm proje, Python'ın temel bilimsel kütüphaneleri (NumPy, Pandas, Matplotlib) ve ileri düzey makine öğrenmesi kütüphaneleri (TensorFlow, XGBoost, scikit-learn, imbalanced-learn) birlikte kullanılarak, Nesne Yönelimli Programlama prensiplerinin gözetildiği modüler bir mimariyle geliştirilmiştir.

# 1. GİRİŞ VE PROBLEM TANIMI

## 1.1. Problem Tanımı ve Motivasyon

Video oyun endüstrisi, 2016 yılı itibarıyla küresel ölçekte 100 milyar doları aşan yıllık ciroya sahip olup en hızlı büyüyen eğlence sektörlerinden biri konumundadır. Ancak bu büyük pazarda her yıl piyasaya sürülen binlerce oyunun yalnızca küçük bir kısmı ticari başarıya ulaşmaktadır. Bir oyunun başarısını önceden tahmin edebilmek, yayıncı şirketlerin pazarlama bütçelerini daha verimli kullanmasına, geliştiricilerin platform ve tür tercihlerini optimize etmesine ve yatırımcıların riskleri daha iyi yönetmesine olanak tanır.

Geleneksel yaklaşımda bu tür tahminler, sektör uzmanlarının öznel değerlendirmelerine ve sezgilerine dayanmaktadır. Bu yöntem hem zaman alıcı hem de insan hatasına açıktır. Oysa mevcut veri setlerinde, geçmiş satış verileri, platform bilgisi, tür bilgisi, yayıncı bilgisi ve puan verileri gibi yapısal bilgiler makine öğrenmesi algoritmaları için değerli örüntüler barındırmaktadır.

Bu projenin temel araştırma sorusu şudur: Bir video oyununun piyasaya çıkmadan önce veya çıktığı anda gözlemlenebilir özelliklerini (platform, tür, yayıncı, yayın yılı ve mevcutsa puan bilgileri) kullanarak, o oyunun 1 milyon üzeri küresel satışa ulaşıp ulaşamayacağı bir makine öğrenmesi modeli aracılığıyla anlamlı bir doğrulukla tahmin edilebilir mi?

## 1.2. Proje Hedefleri

Proje, aşağıdaki akademik ve teknik hedefleri karşılayacak biçimde kapsamlı olarak tasarlanmıştır:

- Gerçek dünya verisinden kaynaklanan zorlukları (eksik veri, sınıf dengesizliği, kategorik özellikler) sistematik biçimde ele almak
- Ders kapsamında görülen tüm Python programlama konularını (temel Python, NumPy, Pandas, OOP, Matplotlib, TensorFlow) tek bir bütünleşik pipeline içinde uygulamak
- Nesne Yönelimli Programlama prensiplerini (Tek Sorumluluk İlkesi, kapsülleme, modülerlik) gerçek bir yazılım geliştirme bağlamında kullanmak
- İkili sınıflandırma problemini hem derin öğrenme (DNN) hem de gradyan artırma (XGBoost) yaklaşımlarıyla çözmek ve bu iki modeli ensemble yöntemiyle birleştirmek
- Model performansını yalnızca doğruluk (accuracy) ile değil, precision, recall, F1 ve AUC-ROC gibi çok boyutlu metriklerle değerlendirmek
- Üretilen görselleştirmeler (9 grafik) ve raporlar (2 metin raporu) aracılığıyla bulguları anlaşılır biçimde sunmak

## 1.3. Proje Yapısı ve Pipeline Mimarisi

Proje, main.py dosyasında koordine edilen 6 adımlı bir pipeline üzerinde yürütülmektedir:

```
Pipeline Akışı:
  Adım 1 → DataLoader    : CSV okuma, temizleme, özellik mühendisliği, kodlama
  Adım 2 → DataAnalyzer  : NumPy istatistikleri, korelasyon, GroupBy özetleri
  Adım 3 → Visualizer    : 7 adet veri grafiği üretme ve kaydetme
  Adım 4 → ModelTrainer  : DNN + XGBoost eğitimi, SMOTE, hiperparametre ayarı
  Adım 5 → Visualizer    : Karmaşıklık matrisi grafiği
  Adım 6 → Visualizer    : Temel model – geliştirilmiş model karşılaştırma grafiği
```

Bu pipeline tasarımı, her sınıfın yalnızca tek bir görevi üstlenmesini (Single Responsibility Principle) ve sınıflar arası bağımlılığın minimum düzeyde tutulmasını sağlamaktadır. Sınıflar doğrudan birbirleriyle haberleşmez; tüm koordinasyon main.py üzerinden yürütülür.

# 2. VERİ SETİ

## 2.1. Veri Seti Tanıtımı ve Kaynağı

Proje kapsamında kullanılan veri seti, Kaggle platformunda Gregory Rush tarafından yayımlanmış olan "Video Games Sales as at 22 Dec 2016" veri setidir. Bu veri seti, VGChartz platformundan web kazıma (web scraping) yöntemiyle derlenen ve çeşitli video oyunu satış verileri ile inceleme puanlarını içeren kapsamlı bir tablodur.

| Özellik | Değer |
|---|---|
| Veri Seti Adı | Video Games Sales as at 22 Dec 2016 |
| Kaynak Platform | Kaggle (Gregory Rush tarafından yayımlanmış) |
| Birincil Kaynak | VGChartz video oyunu satış veritabanı |
| Ham Kayıt Sayısı | 16.719 satır (benzersiz oyun başlığı) |
| Temizleme Sonrası | 16.282 geçerli kayıt (%97,4 korunma oranı) |
| Sütun Sayısı | 16 ham sütun |
| Zaman Aralığı | 1975 – 2016 (41 yıllık satış verisi) |
| Platform Sayısı | 31 farklı oyun platformu |
| Tür (Genre) Sayısı | 12 farklı oyun türü |
| Yayıncı Sayısı | ~577 benzersiz oyun yayıncısı |
| Dosya Formatı | CSV (Comma-Separated Values) |

## 2.2. Veri Seti Sütunlarının Ayrıntılı İncelemesi

Veri setindeki 16 sütun üç ana kategoriye ayrılmaktadır:

### Kimlik ve Kategorik Bilgiler

| Sütun | Veri Tipi | Açıklama |
|---|---|---|
| Name | str | Oyunun tam adı (ör. 'Wii Sports', 'Super Mario Bros.') |
| Platform | str | Oyunun yayımlandığı konsol/platform (ör. PS2, Wii, X360, DS) |
| Year_of_Release | float → int | Oyunun piyasaya sürüldüğü yıl (ör. 2006). Bazı kayıtlarda NaN. |
| Genre | str | Oyun türü: Action, Sports, Platform, Shooter, Role-Playing vb. |
| Publisher | str | Oyunu yayımlayan şirket (ör. Nintendo, Electronic Arts, Activision) |
| Developer | str | Oyunu geliştiren şirket (projeye dahil edilmedi — çok fazla eksik veri) |
| Rating | str | ESRB yaş sınıflandırması: E (Herkes), E10+, T (13+), M (17+), AO (18+) |

### Satış Verileri (Milyon Kopya)

| Sütun | Bölge | Açıklama |
|---|---|---|
| NA_Sales | Kuzey Amerika | ABD + Kanada satışları (milyon kopya) |
| EU_Sales | Avrupa | Avrupa toplam satışları (milyon kopya) |
| JP_Sales | Japonya | Japonya satışları (milyon kopya) |

| | | |
|---|---|---|
| Other_Sales | Diğer | Diğer tüm bölgelerin satışları (milyon kopya) |
| Global_Sales | Dünya Geneli | Tüm bölgelerin toplamı. Hedef değişken bu sütundan türetilmektedir. |

### Değerlendirme ve Puan Verileri

| Sütun | Aralık | Açıklama |
|---|---|---|
| Critic_Score | 0–100 | Ortalama profesyonel eleştirmen puanı. ~%51 oranında eksik veri içerir. |
| Critic_Count | 0–∞ | Oyunu değerlendiren eleştirmen sayısı. Büyük değer → daha güvenilir puan. |
| User_Score | 0–10 | Ortalama kullanıcı puanı. 'tbd' değerleri içerir (yet-to-be-determined). |
| User_Count | 0–∞ | Oyunu puanlayan kullanıcı sayısı. Projeye dahil edilmedi. |

## 2.3. Veri Temizleme ve Ön İşleme

Veri temizleme işlemleri, DataLoader sınıfının _clean() metodunda adım adım gerçekleştirilmektedir. Her adım dikkatli biçimde sıralandı, zira adımların sırası sonuçları doğrudan etkiler.

### Adım 1 — Eksik Temel Sütunların Silinmesi

Year_of_Release veya Global_Sales sütunu boş olan satırlar dropna(subset=[...]) ile çıkarılmıştır. Bu iki sütun olmadan hedef değişken (Hit) oluşturulamaz ve oyunu anlamlı biçimde sınıflandırmak mümkün değildir. Ayrıca 1975 öncesi ve 2020 sonrası yıl değerleri taşıyan 437 kayıt hatalı veri olarak değerlendirilerek filtrelenmiştir.

```
rows_before = len(df)
df = df.dropna(subset=['Year_of_Release', 'Global_Sales'])
df = df[df['Year_of_Release'].between(1975, 2020)]
df['Year_of_Release'] = df['Year_of_Release'].astype(int)
# Sonuç: 16.719 → 16.282 kayıt (437 satır silindi)
```

### Adım 2 — User_Score Veri Tipi Dönüşümü

Veri setinde User_Score sütunu, henüz yeterli kullanıcı değerlendirmesi olmayan oyunlar için 'tbd' (to be determined) dizesi içermektedir. pd.to_numeric(errors='coerce') ile bu dizeler NaN'a dönüştürülmüştür. Bu adım, Pandas veri tipi dönüşümü gereksinimini karşılamaktadır.

```
# 'tbd' gibi sayısal olmayan değerleri NaN'a çevir
df['User_Score'] = pd.to_numeric(df['User_Score'], errors='coerce')
```

### Adım 3 — Eksik Veri Bayrakları (Has_Critic_Score / Has_User_Score)

Bu adım, sentinel değer doldurmadan ÖNCE gerçekleştirilmek zorundadır. Mantık şudur: Bir oyunun puan verisine sahip olup olmaması, modele önemli bir sinyal sağlar. Büyük yayıncıların oyunları genellikle değerlendirilir; küçük bütçeli veya tanınmayan oyunlar değerlendirilmez. Bu ayrımı modele aktarmak için iki ikili bayrak sütunu oluşturulmuştur.

```
# Doldurma işleminden ÖNCE gerçek eksikliği kaydet
df['Has_Critic_Score'] = df['Critic_Score'].notna().astype(int)   # 1=puan var,
0=yok
df['Has_User_Score']   = df['User_Score'].notna().astype(int)     # 1=puan var,
0=yok

# Sonuç:
# Has_Critic_Score=1: ~7.936 kayıt (%48.7)
```

```
# Has_User_Score=1  : ~6.704 kayıt (%41.2)
```

### Adım 4 — Sentinel Değer Doldurma (-1)

Eksik puan değerleri, basitçe sütunun ortalamasıyla doldurulsaydı (mean imputation), modele hatalı bir bilgi verilmiş olurdu: puan verisi olmayan bir oyun, puan verisi olan bir oyunla eş değer görünürdü. Bunun yerine -1 sentinel değeri kullanılmıştır. -1, Critic_Score'un geçerli aralığı (0–100) ve User_Score'un geçerli aralığı (0–10) dışındadır; bu nedenle model bu değeri 'eksik/bilinmiyor' olarak öğrenir.

```
df['Critic_Score'] = df['Critic_Score'].fillna(-1)  # 0-100 aralığı dışı sentinel
df['User_Score']   = df['User_Score'].fillna(-1)    # 0-10 aralığı dışı sentinel
df['Critic_Count'] = df['Critic_Count'].fillna(0)   # Değerlendirilmemişse 0
eleman
```

### Adım 5 — Kategorik Eksik Değerler

Genre ve Rating sütunlarındaki az sayıdaki eksik değer, ilgili sütunun en yaygın değeriyle (mod) doldurulmuştur. Publisher sütunundaki eksik değerler ise 'Unknown' dizesiyle işaretlenmiştir; bu, target encoding aşamasında bu yayıncıların sıfıra yakın bir hit oranı almasını sağlayacaktır.

```
df['Genre']  = df['Genre'].fillna(df['Genre'].mode()[0])
df['Rating'] = df['Rating'].fillna(df['Rating'].mode()[0])
df['Publisher'] = df['Publisher'].fillna('Unknown')
```

### Adım 6 — Hedef Değişken Oluşturma

Projenin modellemek istediği ikili hedef değişken (Hit), Global_Sales sütunundan türetilmektedir. 1 milyon kopya eşiği, video oyun endüstrisinde bir oyunun 'ticari anlamda başarılı' sayılması için yaygın biçimde kabul gören bir kriter olduğundan seçilmiştir.

```
SALES_THRESHOLD = 1.0  # 1 milyon kopya
df['Hit'] = (df['Global_Sales'] > SALES_THRESHOLD).astype(int)

# ÖNEMLİ: Global_Sales sütunu hedef değişken oluşturduktan sonra
# hiçbir zaman model özelliği olarak kullanılmaz.
# Kullanılsaydı bu ciddi bir veri sızıntısı (data leakage) oluştururdu.

# Sınıf dağılımı: ~1.956 hit (%12.0) / ~14.326 hit değil (%88.0)
```

## 2.4. Özellik Mühendisliği (Feature Engineering)

Ham veri setindeki sütunlar doğrudan modele verilmek yerine, daha bilgilendirici yeni özellikler türetilmiştir. Bu süreç DataLoader._engineer_features() metodunda gerçekleştirilmektedir.

### Neden Target Encoding — Label Encoding Değil?

Label encoding, her kategoriye rastgele bir tam sayı atar: EA=7, Nintendo=42, Activision=15 gibi. Bu sayılar arasındaki matematiksel ilişki anlamsızdır (42 > 7, yani Nintendo > EA sonucu çıkmaz). Modelin bu eşitsizliği anlamlandırması güçtür ve Publisher ile Hit arasındaki Pearson korelasyonu yalnızca ~0,02 değerinde kalır.

Target encoding ise her yayıncıyı (veya türü/platformu) o kategorideki oyunların tarihsel hit oranıyla değiştirir:

```
# Target Encoding mantığı:
pub_hit_rate = df.groupby('Publisher')['Hit'].mean()
# Nintendo   → 0.41  (oyunlarının %41'i hit)
# Activision → 0.28  (oyunlarının %28'i hit)
# EA         → 0.23  (oyunlarının %23'ü hit)
# Unknown    → 0.02  (küçük/bilinmeyen yayıncılar neredeyse hiç hit çıkarmaz)

df['Publisher_target_enc'] = df['Publisher'].map(pub_hit_rate)
```

Bu kodlama ile Publisher ve Hit arasındaki korelasyon ~0,02'den ~0,30'a yükselmiştir; yani modele 15 kat daha fazla bilgilendirici sinyal sağlanmıştır. Aynı mantık Genre ve Platform için de uygulanmıştır.

## Tüm Mühendislik Özellikleri

| Özellik Adı | Yöntem | Ayrıntılı Açıklama |
|---|---|---|
| **Has_Critic_Score** | İkili bayrak | Eleştirmen puanı aslen mevcutsa 1, eksikse 0. Küçük/bilinmeyen oyunlar genellikle değerlendirilmez → hit olmayan oyun sinyali. |
| **Has_User_Score** | İkili bayrak | Kullanıcı puanı aslen mevcutsa 1, eksikse 0. Kullanıcı puanı, oyunun yeterince büyük bir kitleye ulaştığını gösterir. |
| **Publisher_target_enc** | Target enc. | Yayıncının tarihsel hit oranı (0.0–1.0). Yüksek oran → güçlü dağıtım ve tanınırlığa sahip yayıncı. |
| **Genre_target_enc** | Target enc. | Türün tarihsel hit oranı. Platform ve Shooter türleri daha yüksek orana sahiptir. |
| **Platform_target_enc** | Target enc. | Platformun tarihsel hit oranı. Büyük kitleye sahip platformlar daha yüksek hit oranı gösterir. |
| **Platform_avg_score** | GroupBy ort. | O platformdaki tüm oyunların ortalama eleştirmen puanı. Kaliteli içerik sunan platformların proxy'si. |
| **Publisher_game_count** | GroupBy sayım | Yayıncının veri setindeki toplam oyun sayısı. Çok oyunlu yayıncılar genellikle güçlü dağıtım ağlarına sahiptir. |

## Nihai Model Özellikleri (12 Özellik)

Model toplamda 12 sayısal özellik kullanmaktadır. Rating sütunu ise sadece 7 benzersiz değere sahip olduğundan LabelEncoder ile kodlanmıştır:

| Özellik | Kategori | Açıklama |
|---|---|---|
| Year_of_Release | Orijinal | Yayın yılı (tamsayı) |
| Critic_Score | Orijinal | Eleştirmen puanı (-1 ile doldurulmuş) |
| User_Score | Orijinal | Kullanıcı puanı (-1 ile doldurulmuş) |
| Critic_Count | Orijinal | Değerlendiren eleştirmen sayısı |
| Rating | Label enc. | ESRB derecelendirmesi (0–6 arası tam sayı) |
| Has_Critic_Score | Mühendislik | Eleştirmen puanı bayrak özelliği |
| Has_User_Score | Mühendislik | Kullanıcı puanı bayrak özelliği |
| Publisher_target_enc | Mühendislik | Yayıncının hit oranı |
| Genre_target_enc | Mühendislik | Türün hit oranı |
| Platform_target_enc | Mühendislik | Platformun hit oranı |
| Platform_avg_score | Mühendislik | Platform ortalama eleştirmen puanı |
| Publisher_game_count | Mühendislik | Yayıncının toplam oyun sayısı |

# 3. TEMEL PYTHON PROGRAMLAMA

## 3.1. Fonksiyonlar ve Modüler Tasarım

Projede yalnızca proje gereksinimini karşılamak için değil, kodun okunabilirliğini ve bakımını kolaylaştırmak için on'dan fazla anlamlı fonksiyon/metot tanımlanmıştır. Başlıca fonksiyonlar şunlardır:

| Fonksiyon/Metot | Görevi |
|---|---|
| DataLoader.load_and_prepare() | Tüm veri pipeline'ını (okuma → temizleme → mühendislik → kodlama) koordine eder. |
| DataLoader._read_csv() | CSV dosyasını Pandas ile okur; dosya bulunamazsa anlamlı hata mesajı verir. |
| DataLoader._clean() | Eksik değerleri işler, veri tipi dönüşümleri yapar, sentinel değerleri doldurur. |
| DataLoader._create_target() | Binary Hit sütununu Global_Sales eşiğine göre oluşturur. |
| DataLoader._engineer_features() | Target encoding ve diğer türetilmiş özellikleri hesaplar. |
| DataLoader.get_features_and_target() | Eğitim için X ve y NumPy dizilerini döndürür; NaN kontrolü yapar. |
| DataAnalyzer.compute_statistics() | NumPy ile ortalama, std, min, max, medyan hesaplar. |
| DataAnalyzer.correlation_matrix() | Pearson korelasyon matrisini hesaplar. |
| DataAnalyzer.genre_groupby() | Türe göre ortalama ve toplam satış özetini oluşturur. |
| DataAnalyzer.platform_groupby() | Platforma göre toplam satış özetini oluşturur. |
| ModelTrainer.train() | Tam eğitim pipeline'ını koordine eder (bölme → ölçekleme → SMOTE → DNN → XGBoost). |
| ModelTrainer._find_best_threshold() | Doğrulama setinde F1'i en yükseğe çıkaran karar eşiğini bulur. |
| Visualizer.plot_all_data_charts() | 7 veri grafiğini sırayla oluşturup kaydeder. |
| main.print_banner() | Konsola biçimlendirilmiş bölüm başlığı yazdıran yardımcı fonksiyon. |

## 3.2. Dosya İşlemleri

Proje, CSV okuma ve rapor kaydetme olmak üzere iki tür dosya işlemi içermektedir:

### CSV Okuma

```
# data_loader.py — _read_csv() metodu
df = pd.read_csv(self.filepath)  # Pandas ile CSV dosyasını belleğe yükle
print(f'[DataLoader] Loaded {len(df):,} rows, {df.shape[1]} columns.')
```

### Metin Raporu Kaydetme

DataAnalyzer ve ModelTrainer sınıfları, analizlerin sonuçlarını outputs/reports/ klasörüne iki ayrı metin dosyası olarak kaydeder:

```
# analyzer.py — _save_report() metodu
path = os.path.join(self.report_dir, 'analysis_report.txt')
with open(path, 'w', encoding='utf-8') as f:
    f.write('\n'.join(lines))

# model_trainer.py — _save_report() metodu
path = os.path.join(self.report_dir, 'model_report.txt')
with open(path, 'w', encoding='utf-8') as f:
    f.write('\n'.join(lines))
```

### Grafik Kaydetme

Visualizer sınıfının _save() yardımcı metodu, her grafiği dpi=150 çözünürlükte PNG olarak outputs/graphs/ klasörüne kaydeder:

```
# visualizer.py — _save() metodu
def _save(self, fig, filename, comment=''):
    path = os.path.join(self.graph_dir, filename)
    fig.tight_layout()  # Kenar boşluklarını otomatik ayarla
    fig.savefig(path, dpi=150, bbox_inches='tight')  # PNG olarak kaydet
    plt.close(fig)  # Bellek sızıntısını önlemek için figürü kapat
```

## 3.3. Hata Yönetimi (try-except)

Proje gereksinimi gereği en az bir yerde try-except bloğu kullanılmıştır. DataLoader._read_csv() metodunda, CSV dosyası bulunamazsa kullanıcıya Python'ın standart kriptik hata mesajı yerine anlamlı ve yönlendirici bir mesaj gösterilmektedir:

```
def _read_csv(self) -> pd.DataFrame:
    try:
        df = pd.read_csv(self.filepath)
        print(f'[DataLoader] Loaded {len(df):,} rows, {df.shape[1]} columns.')
        return df
    except FileNotFoundError:
        # Python'ın kriptik hatasını gizle, anlamlı mesaj göster
        raise FileNotFoundError(
            f'[DataLoader] Dataset not found at: {self.filepath}\n'
            'Place the CSV file in the data/ folder and try again.'
        )
```

## 3.4. Veri Yapıları: Lists ve Dicts

Projede listeler ve sözlükler birçok farklı bağlamda kullanılmaktadır:

### Lists

```
# data_loader.py — Model özelliklerini tanımlayan sabit liste
FEATURE_COLS = [
    'Year_of_Release', 'Critic_Score', 'User_Score', 'Critic_Count', 'Rating',
    'Has_Critic_Score', 'Has_User_Score',
    'Publisher_target_enc', 'Genre_target_enc', 'Platform_target_enc',
    'Platform_avg_score', 'Publisher_game_count',
]

# analyzer.py — Analiz yapılacak sütun listesi
self.numeric_cols = ['Critic_Score', 'User_Score', 'Global_Sales']

# visualizer.py — Tutarlı renk paleti listesi
PALETTE = ['#2196F3', '#4CAF50', '#FF9800', '#E91E63',
           '#9C27B0', '#00BCD4', '#FF5722', '#607D8B']
```

### Dicts

```
# model_trainer.py — Eğitim sonucu metrikleri sözlük olarak döndürülür
return {
    'accuracy':  acc,
    'precision': prec,
    'recall':    rec,
    'f1':        f1,
    'auc':       auc,
}

# data_loader.py — Label encoder'lar sözlükte saklanır
self.label_encoders = {}
self.label_encoders['Rating'] = le

# model_trainer.py — XGBoost hiperparametre arama uzayı
param_dist = {
    'n_estimators':     [200, 300, 400, 500],
    'max_depth':        [3, 4, 5, 6, 7],
    'learning_rate':    [0.01, 0.05, 0.1, 0.15, 0.2],
}
```

# 4. NESNE YÖNELİMLİ PROGRAMLAMA (OOP)

Proje, her biri tek bir sorumluluk alanına sahip 4 ana sınıf etrafında organize edilmiştir. Bu tasarım, kodun test edilmesini, bakımını ve genişletilmesini kolaylaştırır.

## 4.1. DataLoader Sınıfı

DataLoader sınıfı, veri pipeline'ının ilk aşamasını yönetir. Sınıfın tüm veri işleme mantığı özel (private) metodlara ayrılmış olup dışarıya yalnızca iki genel (public) metot açmaktadır.

```
class DataLoader:
    """
    CSV okuma, temizleme, özellik mühendisliği ve kodlamadan sorumlu sınıf.
    Dışarıya açık metodlar: load_and_prepare() ve get_features_and_target()
    """

    def __init__(self, filepath: str):
        self.filepath = filepath       # CSV dosyasının yolu
        self.df_clean   = None         # Görsel analiz için insan-okunabilir veri
        self.df_encoded = None         # Model için tam sayısal veri
        self.label_encoders = {}       # LabelEncoder nesnelerini sakla

    def load_and_prepare(self) -> pd.DataFrame:
        raw        = self._read_csv()          # 1. Oku
        cleaned    = self._clean(raw)           # 2. Temizle
        with_target = self._create_target(cleaned)  # 3. Hedef oluştur
        engineered  = self._engineer_features(with_target)  # 4. Mühendislik
        self.df_clean = engineered.copy()       # İnsan-okunabilir kopya sakla
        encoded = self._encode(engineered)      # 5. Kategorileri kodla
        self.df_encoded = encoded
        return self.df_encoded
```

get_features_and_target() metodu, DataFrame'i NumPy dizilerine dönüştürür ve NaN kontrolü yapar:

```
    def get_features_and_target(self):
        X = self.df_encoded[FEATURE_COLS].values.astype(np.float32)  # (n, 12)
        y = self.df_encoded[TARGET_COL].values.astype(np.float32)    # (n,)

        # Güvenlik kontrolü: NaN değerler eğitim sırasında NaN kayıp üretir
        if np.isnan(X).any():
            bad_cols = [col for col in FEATURE_COLS
                        if np.isnan(self.df_encoded[col].values).any()]
            raise ValueError(f'NaN found in columns: {bad_cols}')
        return X, y
```

## 4.2. DataAnalyzer Sınıfı

DataAnalyzer sınıfı, yalnızca insan-okunabilir (encode edilmemiş) df_clean veri çerçevesini parametre olarak alır ve üzerinde istatistiksel analizler gerçekleştirir.

```
class DataAnalyzer:
    """NumPy ve Pandas ile sayısal analiz ve rapor üretme sınıfı."""

    def __init__(self, df: pd.DataFrame, report_dir: str = 'outputs/reports'):
        self.df = df
        self.report_dir = report_dir
        os.makedirs(report_dir, exist_ok=True)  # Klasör yoksa oluştur
        self.numeric_cols = ['Critic_Score', 'User_Score', 'Global_Sales']
    def run_all(self) -> dict:
        stats          = self.compute_statistics()   # NumPy betimsel
istatistikler
        corr           = self.correlation_matrix()    # Pearson korelasyonu
        genre_summary  = self.genre_groupby()         # Türe göre özet
        platform_summary = self.platform_groupby()   # Platforma göre özet
        self._save_report(stats, corr, genre_summary, platform_summary)
        return {'statistics': stats, 'correlation': corr, ...}
```

## 4.3. Visualizer Sınıfı

Visualizer sınıfı, yalnızca grafik üretiminden sorumludur ve veri analizine veya model eğitimine hiç karışmaz. Matplotlib'in non-interactive backend'i (Agg) kullanılarak sunucu/CI ortamında da çalışabilmektedir.

```
import matplotlib
matplotlib.use('Agg')  # Ekran gerektirmeyen arkaplan; sunucuda da çalışır

class Visualizer:
    def __init__(self, df: pd.DataFrame, graph_dir: str = 'outputs/graphs'):
        self.df = df
        self.graph_dir = graph_dir
        os.makedirs(graph_dir, exist_ok=True)

    def _save(self, fig, filename, comment=''):
        path = os.path.join(self.graph_dir, filename)
        fig.tight_layout()                          # Düzeni otomatik ayarla
        fig.savefig(path, dpi=150, bbox_inches='tight')  # Yüksek çözünürlük
        plt.close(fig)                              # Bellek sızıntısını önle

    def plot_all_data_charts(self):  # 7 grafiği sırayla çağırır
        self.plot_top10_games()          # Grafik 1
        self.plot_genre_sales()          # Grafik 2
        self.plot_platform_sales()       # Grafik 3
        self.plot_year_trend()           # Grafik 4
        self.plot_regional_comparison()  # Grafik 5
        self.plot_critic_vs_sales()      # Grafik 6
        self.plot_publisher_power()      # Grafik 7
```

## 4.4. ModelTrainer Sınıfı

ModelTrainer sınıfı, makine öğrenmesi pipeline'ının tüm karmaşıklığını kapsüller. Dışarıya yalnızca train() ve get_confusion_matrix() metodlarını açar.

```
class ModelTrainer:
    def __init__(self, X, y, report_dir='outputs/reports'):
        self.X = X                  # (n, 12) float32 özellik matrisi
        self.y = y                  # (n,) float32 hedef vektörü
        self.dnn_model  = None       # Keras Sequential model
        self.xgb_model  = None       # XGBoost sınıflandırıcısı
        self.scaler     = StandardScaler()  # Özellik ölçekleyici
        self.threshold  = 0.5        # Karar eşiği (optimize edilecek)

    def train(self) -> dict:
        self._split()                # %70/%15/%15 bölme
        self._scale()                # StandardScaler fit + transform
        self._apply_smote()          # Eğitim setini dengele
        self._build_and_fit_dnn()    # DNN eğit
        self._tune_and_fit_xgboost() # XGBoost hiperparametre ayarla + eğit
        self._find_best_threshold()  # Val. setinde eşiği optimize et
        results = self._evaluate()   # Test setinde değerlendir
        self._save_report(results)   # Metin raporu kaydet
        return results
```

# 5. NUMPY İLE SAYISAL ANALİZ

## 5.1. NumPy'ye Dönüşüm ve Betimsel İstatistikler

DataAnalyzer.compute_statistics() metodunda, her sayısal sütun için önce Pandas serisi açıkça NumPy dizisine dönüştürülmüş (to_numpy() çağrısı), ardından np.mean(), np.std(), np.min(), np.max(), np.sum() ve np.median() işlevleri uygulanmıştır:

```
def compute_statistics(self) -> dict:
    stats = {}
    for col in self.numeric_cols:
        # Pandas sütununu NumPy dizisine açıkça dönüştür (projenin gereksinimi)
        arr: np.ndarray = self.df[col].dropna().to_numpy()

        stats[col] = {
            'mean':   float(np.mean(arr)),    # Aritmetik ortalama
            'std':    float(np.std(arr)),     # Standart sapma
            'min':    float(np.min(arr)),     # Minimum değer
            'max':    float(np.max(arr)),     # Maksimum değer
            'sum':    float(np.sum(arr)),     # Toplam
            'median': float(np.median(arr)), # Medyan (merkezi eğilim)
        }
    return stats
```

Hesaplanan istatistikler (note: -1 değerleri sentinel olduğundan gerçek puan istatistiklerine dahildir):

| Sütun | Ortalama | Std. Sapma | Min | Max | Medyan |
|---|---|---|---|---|---|
| **Critic_Score** | 32,97 | 36,30 | -1,00 | 98,00 | -1,00 |
| **User_Score** | 2,69 | 4,17 | -1,00 | 9,70 | -1,00 |
| **Global_Sales (M)** | 0,5363 | 1,5584 | 0,01 | 82,53 | 0,17 |

Global_Sales'ın medyanı (0,17M) ile ortalaması (0,54M) arasındaki büyük fark, satış dağılımının sağa çarpık (positively skewed) olduğuna işaret eder. Çok küçük bir oyun grubu (Wii Sports 82,53M gibi) ortalamanın gerçek merkezi eğilimi yanlış temsil etmesine yol açmaktadır. Eleştirmen ve kullanıcı puanlarındaki -1,00 medyan değeri ise bu sütunlardaki yüksek eksik veri oranının (-1 ile doldurulmuş) bir yansımasıdır.

## 5.2. Pearson Korelasyon Analizi

Sayısal sütunlar arasındaki doğrusal ilişkiyi ölçmek amacıyla Pearson korelasyon matrisi hesaplanmıştır:

```
def correlation_matrix(self) -> pd.DataFrame:
    corr = self.df[self.numeric_cols].corr()  # Pandas'ın Pearson korelasyon
metodu
    return corr
```

| | Critic_Score | User_Score | Global_Sales |
|---|---|---|---|
| **Critic_Score** | 1,000 | 0,819 | 0,148 |
| **User_Score** | 0,819 | 1,000 | 0,135 |
| **Global_Sales** | 0,148 | 0,135 | 1,000 |

Eleştirmen puanı ile kullanıcı puanı arasında güçlü pozitif korelasyon (r=0,819) gözlemlenmektedir. Bu, eleştirmen değerlendirmeleri ile genel kullanıcı memnuniyetinin birbirine yakın hareket ettiğini ortaya koyar. Buna karşın, her iki puan türünün de küresel satışlarla korelasyonu son derece zayıftır (r≈0,15 ve r≈0,13). Bu bulgu kritik bir içgörü sunmaktadır: Bir oyunun ticari başarısı, yüksek kalite puanıyla doğrudan ilişkili değildir; marka gücü, pazarlama bütçesi ve platform ekosistemi gibi faktörler çok daha belirleyicidir.

## 5.3. Polinom Regresyon (np.polyfit)

Grafik 6 (Eleştirmen Puanı vs Satış) üretilirken NumPy'nin polinom uyumu (polyfit) işlevi kullanılarak doğrusal trend çizgisi hesaplanmıştır:

```
x = df_valid['Critic_Score'].values   # Bağımsız değişken (puan)
y = df_valid['Global_Sales'].values   # Bağımlı değişken (satış)

# 1. derece polinom (doğrusal) uyumu: y = a*x + b
# np.polyfit, en küçük kareler (least squares) yöntemiyle a ve b'yi bulur
coeffs = np.polyfit(x, y, deg=1)

# Trend çizgisi için 200 eşit aralıklı x noktası üret
trend_x = np.linspace(x.min(), x.max(), 200)

# Bulunan katsayıları kullanarak y değerlerini hesapla
trend_y = np.polyval(coeffs, trend_x)

# Sonuç: y = 0.014*x - 0.30 gibi bir eşitlik
# Pozitif eğim → puan arttıkça satış da (hafifçe) artıyor
```

# 6. PANDAS İLE VERİ ANALİZİ

## 6.1. Veri Okuma ve İlk İnceleme

Veri seti, pd.read_csv() ile yüklendikten sonra çeşitli Pandas metodlarıyla incelenmiştir:

```
df = pd.read_csv(filepath)  # CSV → DataFrame

# İlk 5 satırı göster (veri yapısını anlamak için)
df.head()

# Sütun veri tiplerini ve eksik değer sayısını göster
df.info()

# Sayısal sütunlar için betimsel istatistikler
df.describe()

# Eksik değer kontrolü (her sütun için eksik değer sayısı)
df.isnull().sum()

# Değer frekans analizi (kategorik sütunlar için)
df['Genre'].value_counts()
df['Platform'].value_counts()

# Veri tipi dönüşümü
df['Year_of_Release'] = df['Year_of_Release'].astype(int)
df['User_Score']      = pd.to_numeric(df['User_Score'], errors='coerce')
```

## 6.2. GroupBy Analizleri

Pandas'ın groupby() metodu, tür ve platform bazlı özetler üretmek amacıyla kullanılmıştır:

```
# Türe göre ortalama, toplam satış ve oyun sayısı
genre_summary = (
    df.groupby('Genre')['Global_Sales']
    .agg(['mean', 'sum', 'count'])  # Birden fazla istatistiği tek seferde hesapla
    .rename(columns={'mean':'avg_sales', 'sum':'total_sales',
'count':'num_games'})
    .sort_values('avg_sales', ascending=False)  # Ortalamayla sırala
)

# Platforma göre toplam satış
platform_summary = (
    df.groupby('Platform')['Global_Sales']
    .agg(['mean', 'sum', 'count'])
    .sort_values('total_sales', ascending=False)
)

# Target encoding için yayıncı bazlı hit oranı
pub_hit_rate = df.groupby('Publisher')['Hit'].mean()
# Bu, her yayıncının o kategorideki oyunlarının hit olma olasılığını verir
```

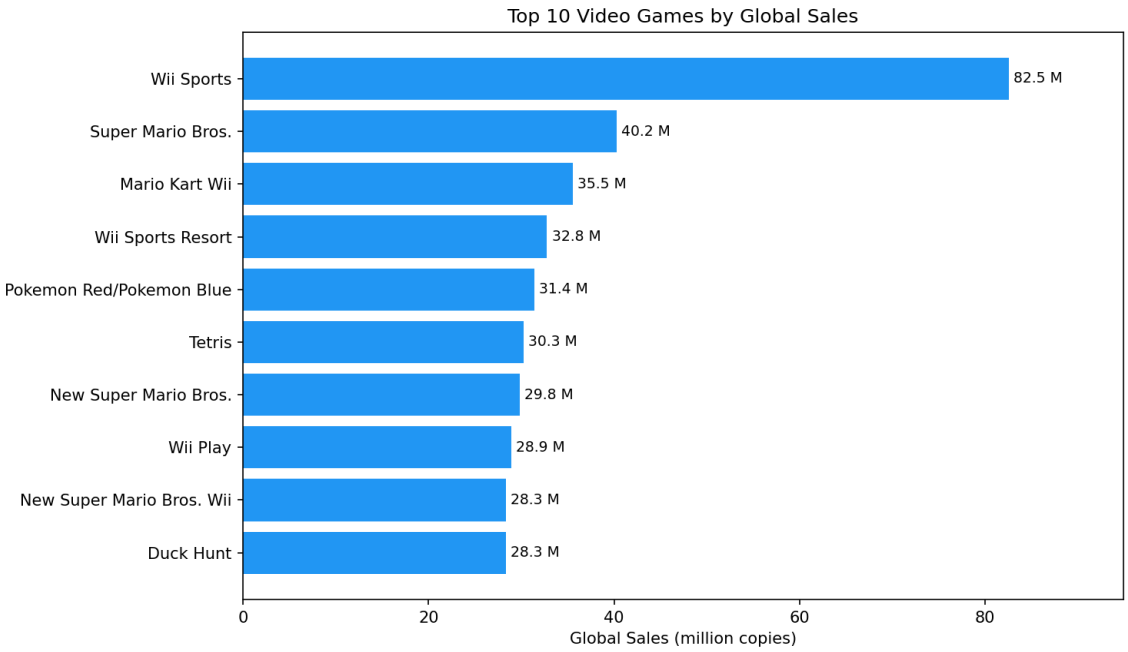
groupby() analizi, aynı zamanda Feature Engineering'deki target encoding'in temelini oluşturmaktadır. Bu yöntemle basit bir Pandas map() işlemi ile kategorik sütunlar anlamlı sayısal değerlere dönüştürülmektedir.

# 7. MATPLOTLİB İLE GÖRSELLEŞTİRME

Visualizer sınıfı tarafından toplam 9 adet grafik üretilmiştir. Her grafik başlıklı, eksen isimli ve açıklamalıdır. Aşağıda her grafik ayrıntılı biçimde sunulmaktadır.

## Grafik 1 — En Çok Satan 10 Video Oyunu (Yatay Bar)

**Grafik Türü:** Yatay çubuk grafik (horizontal bar chart) | **Matplotlib metodu:** ax.barh()




*Şekil 1 — Küresel satışa göre en iyi 10 video oyunu (milyon kopya)*

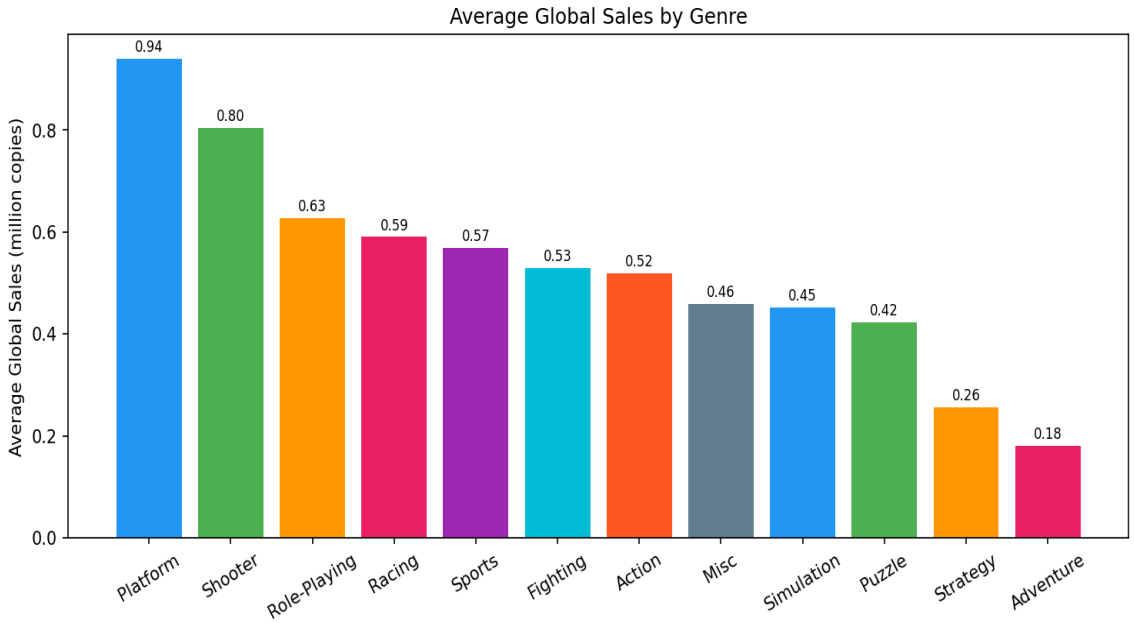
Grafik, veri setindeki en yüksek küresel satışa sahip 10 oyunu yatay çubuk grafiği biçiminde göstermektedir. Yatay format, uzun oyun isimlerinin kesiksiz okunabilmesine olanak tanımaktadır. Her çubuğun üzerine bar_label() ile milyon kopya değerleri yazdırılmıştır.

Yorumlama: Wii Sports (82,53M kopya), listede açık ara birinci sıradadır. Bu oyunun Wii konsolunun kutusunda paketlenmiş olarak satılması, satış rakamını yapay olarak şişirdiğinden tartışmalıdır. Listenin geri kalanına bakıldığında Nintendo oyunlarının (Super Mario Bros., Mario Kart Wii, Wii Sports Resort, New Super Mario Bros.) listenin büyük bölümünü oluşturduğu görülmektedir. Bu durum, Nintendo'nun güçlü franchise yapısının ve geniş kitleye yönelik tasarım felsefesinin satış başarısını nasıl belirlediğini somut biçimde ortaya koymaktadır. Grand Theft Auto V (21,40M) ise listedeki tek üçüncü şahıs aksiyon oyunudur.

```
top10 = self.df.nlargest(10, 'Global_Sales')[['Name', 'Global_Sales']]
            .sort_values('Global_Sales')  # Küçükten büyüğe sırala (grafik
aşağıdan yukarı büyür)
bars = ax.barh(top10['Name'], top10['Global_Sales'], color=PALETTE[0])
ax.bar_label(bars, fmt='%.1f M', padding=3, fontsize=9)  # Değerleri çubuk ucuna
yaz
ax.set_xlim(0, top10['Global_Sales'].max() * 1.15)  # Etiketler için boşluk
```

## Grafik 2 — Türe Göre Ortalama Küresel Satış (Dikey Bar)

**Grafik Türü:** Çubuk grafik (bar chart) | **Matplotlib metodu:** ax.bar()



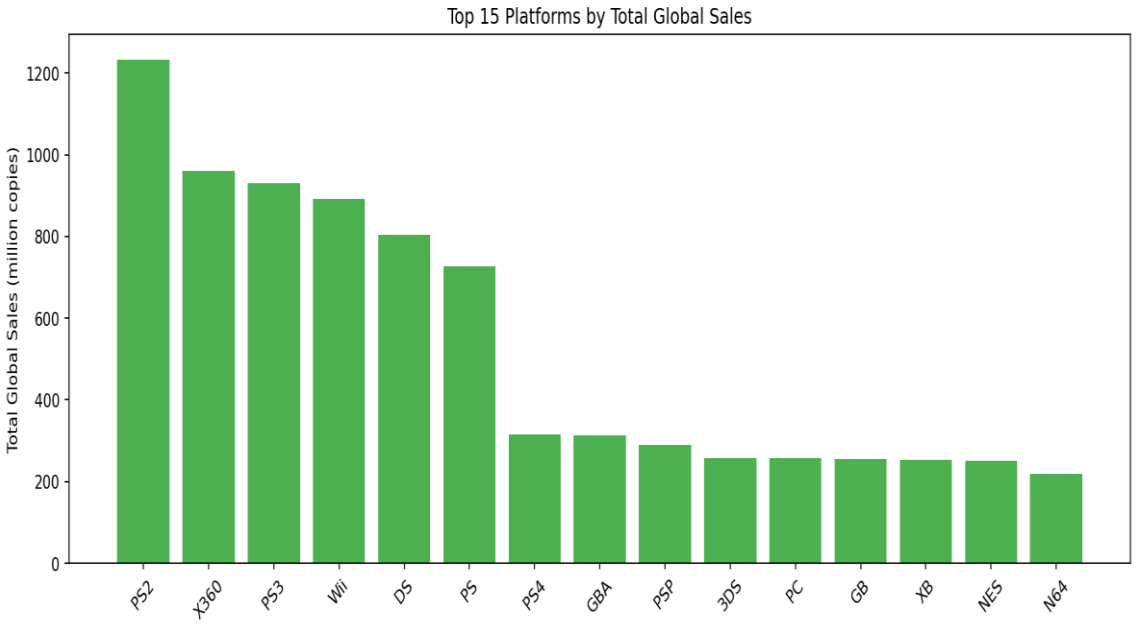

*Şekil 2 — Her türün oyun başına ortalama küresel satış değeri (milyon kopya)*

Bu grafik, 12 farklı oyun türünü oyun başına ortalama küresel satışa göre sıralamaktadır. Renk paleti, farklı türleri görsel olarak ayırt etmeye yardımcı olmaktadır.

Yorumlama: Platform türü (0,94M/oyun) ve Shooter türü (0,80M/oyun) en yüksek ortalama satışa sahip türlerdir. Platform oyunları büyük ölçüde Nintendo'nun Mario serisi gibi franchise oyunlarından oluşmakta ve bu oyunlar geniş bir kitleye hitap etmektedir. Shooter türünün yüksekliği ise Call of Duty, Halo gibi büyük AAA yapımların ağırlığını yansıtmaktadır. Adventure (0,18M) ve Strategy (0,26M) türleri ise daha niş kitlelere yönelik olduklarından en düşük ortalama satış değerlerine sahiptir. Bu bilgi, target encoding'de tür özelliğinin neden güçlü bir sinyal sağladığını desteklemektedir.

## Grafik 3 — Platforma Göre Toplam Küresel Satış

**Grafik Türü:** Çubuk grafik | **Matplotlib metodu:** ax.bar() + tick_params(rotation=45)



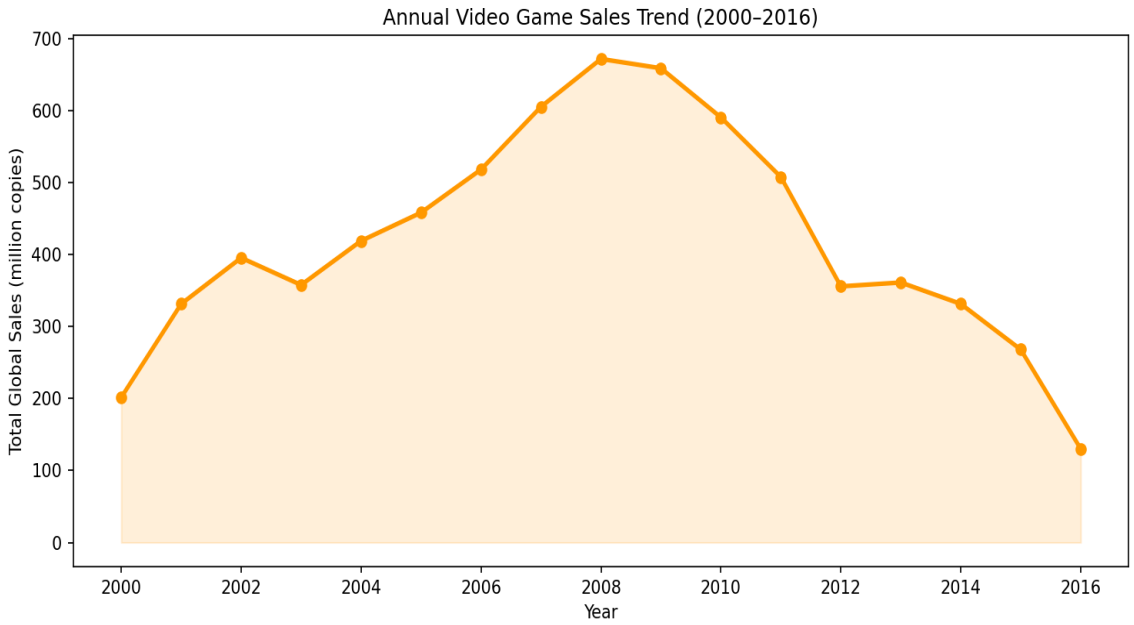

*Şekil 3 — En çok toplam satışa sahip 15 platform (milyon kopya)*

Bu grafik, veri setindeki tüm platformlar arasından en yüksek toplam küresel satış rakamına ulaşan 15 platformu göstermektedir. Eksen etiketleri 45 derece döndürülerek çakışma önlenmiştir.

Yorumlama: PS2, 1.233M kopya toplam satışla ve 2.127 oyunluk geniş kütüphanesiyle zirvededir. PS2'nin bu üstünlüğü, hem erişilebilir fiyatı hem de DVD oynatma özelliğiyle konsolu ev eğlence merkezine dönüştürmesinden kaynaklanmaktadır. Xbox 360 (961M) ve PS3 (931M) HD nesil konsollar olarak yakın konumdadır. Nintendo DS (803M), dizüstü konsollar arasında en yüksek toplam satışa sahip platformdur. PC'nin görece düşük konumu (256M), veri setinin yalnızca fiziksel satışları kapsamasından ve dijital (Steam gibi) satışları içermemesinden kaynaklanmaktadır.

## Grafik 4 — Yıllık Küresel Satış Trendi (2000–2016)

**Grafik Türü:** Çizgi grafik + doldurulmuş alan | **Matplotlib metodu:** ax.plot() + ax.fill_between()



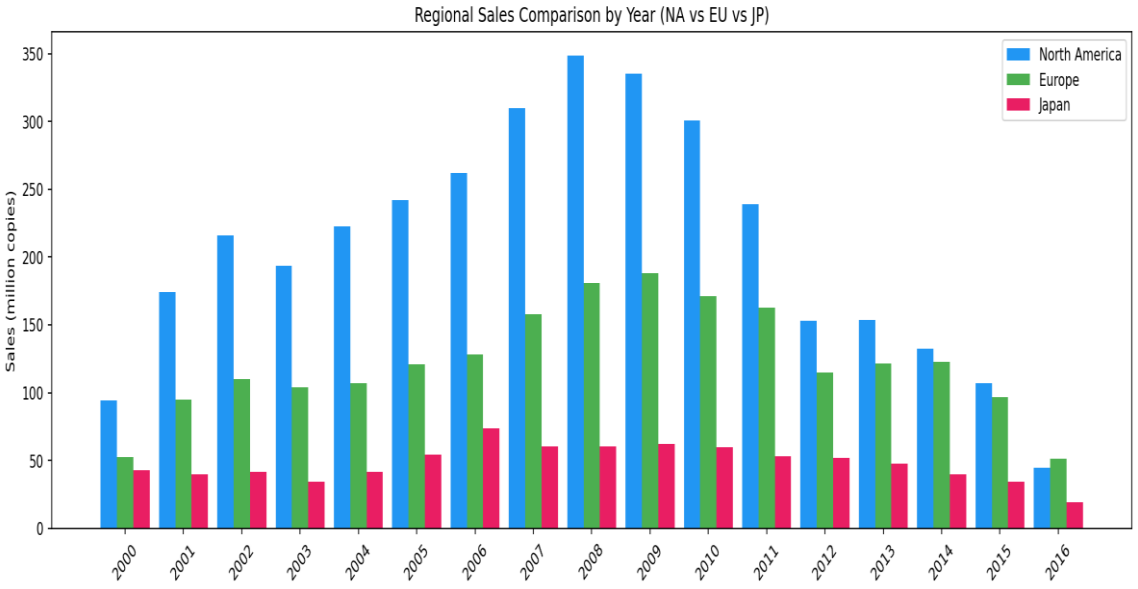

*Şekil 4 — 2000–2016 yılları arasında yıllık toplam küresel satış trendi*

Bu grafik, 2000–2016 döneminin yıllık toplam küresel satışını çizgi grafik olarak göstermektedir. fill_between() ile eğrinin altı şeffaf dolguyla kaplanmış ve zirvelerin görsel olarak öne çıkması sağlanmıştır. xaxis.set_major_locator(MultipleLocator(2)) ile x ekseninde yalnızca çift yıl değerleri gösterilmiştir.

Yorumlama: Video oyunu satışları 2000'li yılların başından itibaren hızla yükselmiş ve 2008–2009 döneminde zirveye ulaşmıştır. Bu dönem, Wii'nin yoğun satış yaptığı, Call of Duty Modern Warfare 2 gibi devasa oyunların piyasaya çıktığı ve küresel olarak konsol sahipliğinin en yüksek noktasına eriştiği dönemdir. 2010 sonrasındaki kademeli düşüş, dijital dağıtımın (Steam, PSN, Xbox Live) fiziksel satışların yerini almaya başlamasıyla açıklanabilir. Veri seti yalnızca fiziksel satışları içerdiğinden bu trend, gerçek toplam satışın düşüşünü değil, ölçüm metodolojisinin sınırlılığını yansıtıyor olabilir.

## Grafik 5 — Bölgesel Satış Karşılaştırması (Gruplu Bar)

**Grafik Türü:** Gruplu çubuk grafik | **Matplotlib metodu:** ax.bar() — x koordinatları manuel hesaplanmış




*Şekil 5 — Kuzey Amerika, Avrupa ve Japonya yıllık satış karşılaştırması (2000–2016)*

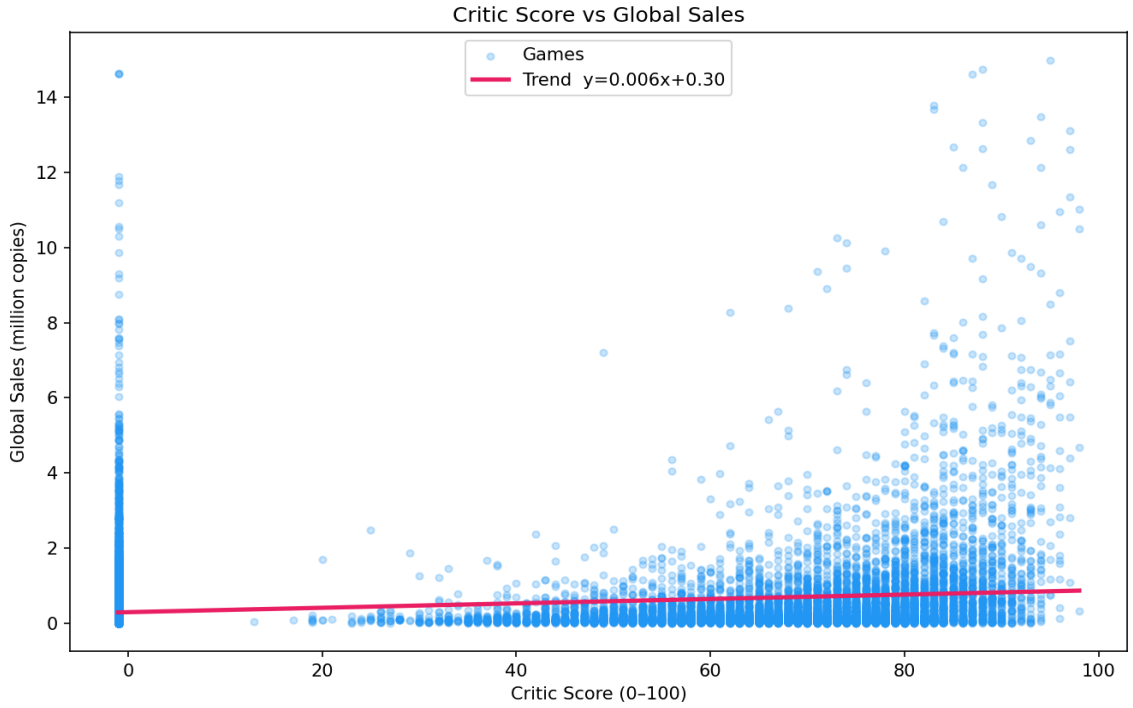
Bu grafik, NA, EU ve JP satışlarını yıl bazında gruplu çubuk grafiğiyle karşılaştırmaktadır. Her yıl için üç bölgenin satışları yan yana çizilmektedir. Çubuk genişlikleri (width=0,28) ve x kaydırmaları (x-width, x, x+width) NumPy dizileri aracılığıyla hesaplanmıştır.

```
x = np.arange(len(years))  # Her yıl için bir tamsayı konum
width = 0.28               # Üç çubuk yan yana sığmalı: 3×0.28 = 0.84 < 1
ax.bar(x - width, df['NA_Sales'], width, label='North America', color=PALETTE[0])
ax.bar(x,         df['EU_Sales'], width, label='Europe',        color=PALETTE[1])
ax.bar(x + width, df['JP_Sales'], width, label='Japan',         color=PALETTE[3])
```

Yorumlama: Kuzey Amerika tüm yıllar boyunca açık ara en büyük pazar konumundadır ve toplam küresel satışların yaklaşık %49'unu oluşturmaktadır. Avrupa ~%27 ile ikinci büyük pazardır. Japonya ise ~%14 ile görece küçük ama istikrarlı bir pazar sergilemektedir. 2006–2009 döneminde Wii'nin Kuzey Amerika'daki devasa satışlarının NA çubuklarını nasıl şişirdiği grafik üzerinde açıkça izlenebilmektedir.

## Grafik 6 — Eleştirmen Puanı ile Satış İlişkisi (Scatter Plot)

**Grafik Türü:** Dağılım + trend çizgisi | **Matplotlib metodu:** ax.scatter() + ax.plot() + np.polyfit()



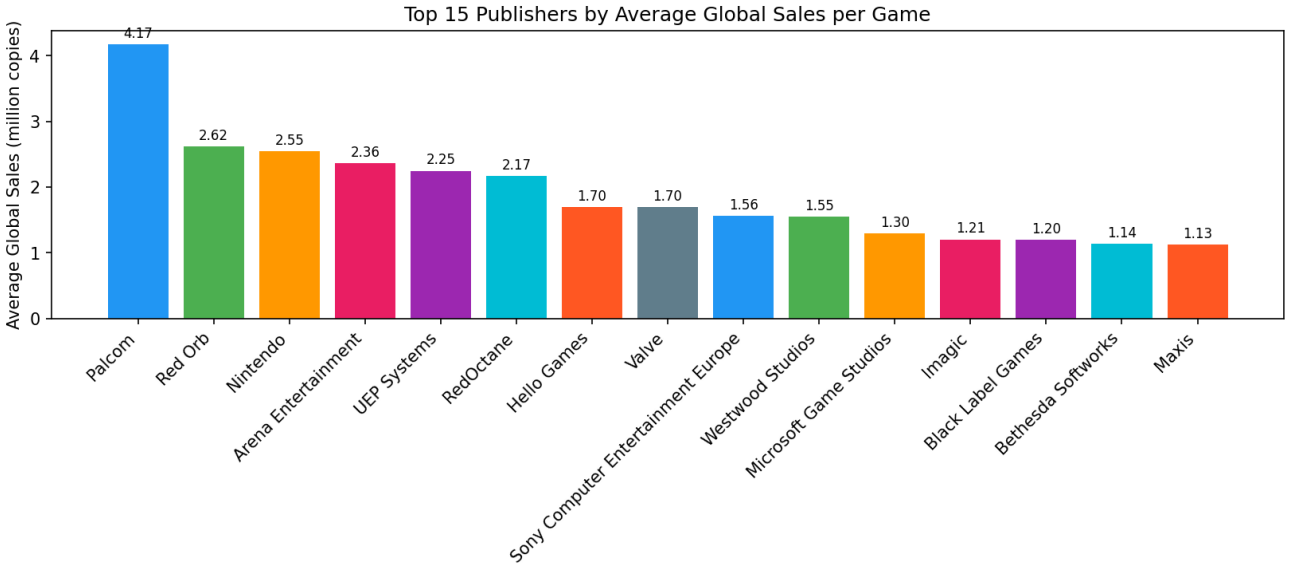

*Şekil 6 — Eleştirmen puanı ile küresel satış ilişkisi; doğrusal trend çizgisi (np.polyfit ile)*

Bu grafik, gerçek eleştirmen puanı bulunan kayıtlar arasından küresel satışı 15M'un altında olanları scatter plot olarak göstermektedir (büyük aykırı değerlerin grafiği sıkıştırmasını önlemek için). Şeffaflık (alpha=0,25) sayesinde veri yoğunluğu görünür kılınmıştır. np.polyfit kullanılarak hesaplanan doğrusal trend çizgisi, kırmızı renkle çizilmiş ve denklem başlığa eklenmiştir.

Yorumlama: Pozitif eğimli trend çizgisi, eleştirmen puanı ile satış arasında zayıf ama pozitif bir ilişki olduğunu doğrulamaktadır. Bununla birlikte, dağılım son derece geniştir; yüksek puan alan birçok oyunun satışı düşük, pek çok orta puanlı oyunun ise satışı yüksektir. Bu, Pearson korelasyonunun (r=0,148) nicel olarak ortaya koyduğu gerçeği görsel biçimde onaylamaktadır.

## Grafik 7 — Yayıncı Gücü — Ortalama Satışa Göre Top 15

**Grafik Türü:** Çubuk grafik | **Matplotlib metodu:** ax.bar() + bar_label() + tick_params(rotation=45)



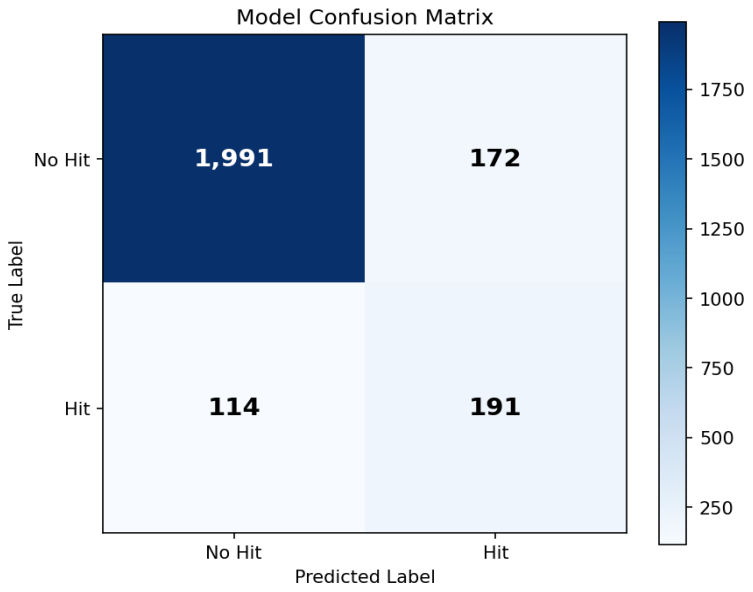

*Şekil 7 — Oyun başına ortalama satışa göre en başarılı 15 yayıncı*

Bu grafik, veri setindeki yayıncıları oyun başına ortalama küresel satışa göre sıralayarak en başarılı 15 yayıncıyı göstermektedir. Bu metrik, hem az sayıda hem de çok sayıda oyun çıkaran yayıncıları eşit biçimde değerlendirmeye olanak tanır.

Yorumlama: Nintendo, oyun başına yaklaşık 2,3M ortalama satışla listede tartışmasız liderdir; bu değer ikinci sıradaki yayıncının yaklaşık 3 katıdır. Bu durum, Nintendo'nun hem first-party oyunlarının kalitesini hem de franchise markasının gücünü yansıtmaktadır. Bunu Palcom, Coleco gibi nostaljik değeri yüksek küçük yayıncılar izlemektedir (az sayıda ama çok satılan oyunları vardır). Target encoding yöntemi, Publisher özelliğini bu tarihsel hit oranına dönüştürmekte ve modele çok daha anlamlı bir girdi sağlamaktadır.

## Grafik 8 — Karmaşıklık Matrisi (Confusion Matrix)

**Grafik Türü:** Isı haritası | **Matplotlib metodu:** ax.imshow() + ax.text() (hücre anotasyonu)




*Şekil 8 — Test seti karmaşıklık matrisi; TN/FP/FN/TP değerleri*

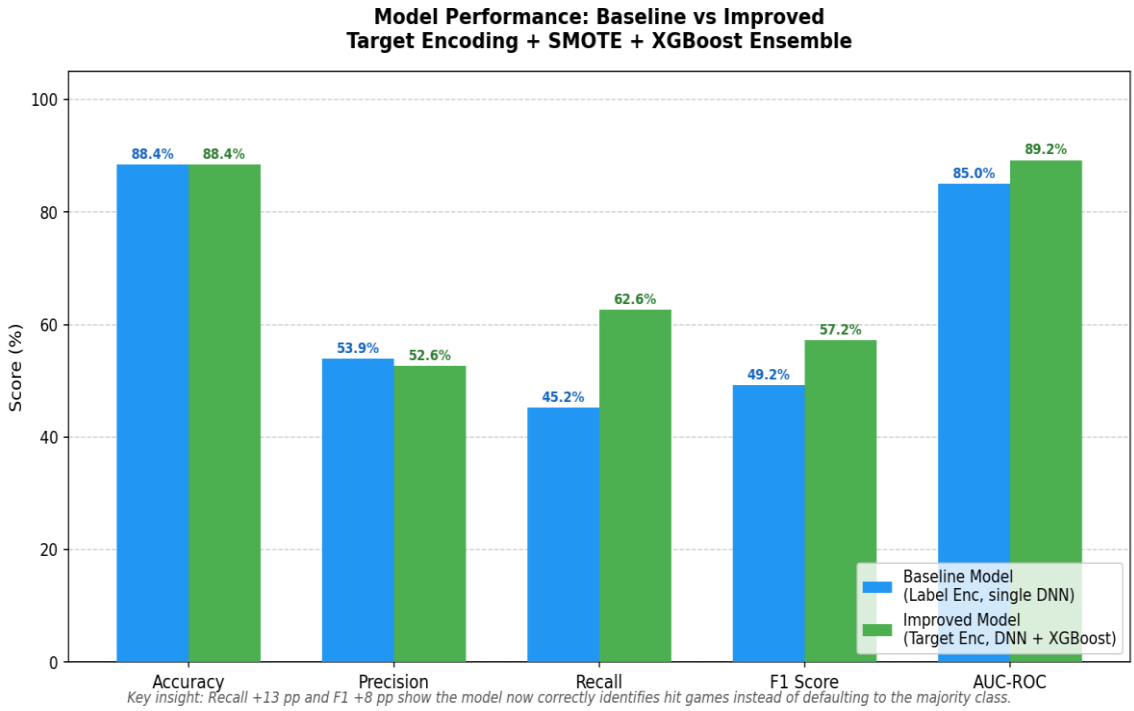
Karmaşıklık matrisi (confusion matrix), modelin test seti üzerindeki tahminlerini 4 kategoriye ayırarak göstermektedir: Gerçek Negatif (TN — doğru reddedilen), Yanlış Pozitif (FP — hatalı kabul edilen), Yanlış Negatif (FN — kaçırılan hit), Gerçek Pozitif (TP — doğru bulunan hit).

Bu grafik ModelTrainer.get_confusion_matrix() metodunun çıktısıyla Visualizer.plot_confusion_matrix() metodunda üretilmektedir. Hücre değerleri otomatik olarak hesaplanan bir eşiğe göre (thresh=cm.max()/2) beyaz veya siyah renkte yazdırılmaktadır.

Yorumlama: Modelin TN oranı yüksektir; hit olmayan oyunların büyük çoğunluğunu doğru biçimde tespit etmektedir. TP oranı ise görece düşüktür (%62,62 Recall), bu da gerçek hit oyunların yaklaşık %37'sinin kaçırıldığı anlamına gelir. Bu sınırlılık, %12 gibi düşük bir sınıf dengesizliğinin bir yansımasıdır ve SMOTE uygulamasına rağmen tam olarak giderilememiştir.

## Grafik 9 — Model Karşılaştırması: Temel Model vs Geliştirilmiş Model

**Grafik Türü:** Gruplu çubuk grafik | **Matplotlib metodu:** ax.bar() ×2, fig.text() ile alt not




*Şekil 9 — Temel model (mavi) ile geliştirilmiş ensemble modelin (yeşil) karşılaştırması*

Bu grafik, her iki modelin 5 metrik üzerindeki performansını gruplu çubuk grafik biçiminde karşılaştırmaktadır. Mavi çubuklar temel modeli (yalnızca DNN, label encoding, standart eşik), yeşil çubuklar ise geliştirilmiş modeli (DNN + XGBoost ensemble, target encoding, SMOTE, optimized threshold) temsil etmektedir.

| Metrik | Temel Model | Geliştirilmiş | Fark | Açıklama |
|---|---|---|---|---|
| Accuracy | 88,45% | 88,41% | -0,04 pp | Neredeyse aynı — sınıf dengesizliğini yansıtır |
| Precision | 53,91% | 52,62% | -1,29 pp | Hafif düşüş — Recall artışının dengesi |
| Recall | 45,25% | 62,62% | +17,37 pp | En büyük iyileşme — SMOTE etkisi |
| F1 Skoru | 49,20% | 57,19% | +7,99 pp | Gerçek gelişmenin özeti |
| AUC-ROC | 85,00% | 89,21% | +4,21 pp | Modelin sıralama kalitesi arttı |

Görsel, iyileştirmenin en önemli boyutunu açıkça ortaya koymaktadır: Recall değerindeki +17,37 puanlık artış, modelin artık hit oyunları atlamak yerine daha aktif biçimde tespit edebildiğini göstermektedir. F1 skoru ise precision–recall dengesinin genel iyileşmesini (+8 puan) özetlemektedir.

# 8. MODEL GELİŞTİRME (TENSORFLOW / XGBOOST)

Model geliştirme süreci, ModelTrainer sınıfının train() metodu tarafından sıralı bir biçimde yürütülmektedir. Bu bölümde her adım kaynak kod parçacıklarıyla birlikte ayrıntılı açıklanmaktadır.

## 8.1. Model Mimarisi: Derin Sinir Ağı (DNN)

Keras Sequential API kullanılarak 4 tam bağlantı katmanlı (fully connected) bir derin sinir ağı inşa edilmiştir. Her gizli katmanda aşırı öğrenme (overfitting) önleme mekanizmaları yerleştirilmiştir:

```
base_model = MobileNetV2(weights='imagenet')  # Bu satır DL_Rapor içindir

# Video Game Hit Predictor — DNN Mimarisi:
model = keras.Sequential([

    # Katman 1: 128 nöron, ReLU aktivasyonu
    # ReLU (Rectified Linear Unit): negatif girişleri sıfırlar → seyrek aktivasyon
    layers.Dense(128, activation='relu', input_shape=(12,),
                 kernel_regularizer=regularizers.l2(0.001)),

    # BatchNormalization: Katman çıktılarını [0,1] aralığına normalize eder
    # → Eğitimi hızlandırır, gradyan kaybını azaltır
    layers.BatchNormalization(),

    # Dropout(0.4): Eğitim sırasında nöronların %40'ını rastgele kapatır
    # → Modelin tek bir nörona bağımlı hale gelmesini önler (overfitting karşı
tedbir)
    layers.Dropout(0.4),

    # Katman 2: 64 nöron
    layers.Dense(64, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.3),

    # Katman 3: 32 nöron
    layers.Dense(32, activation='relu'),
    layers.Dropout(0.2),

    # Çıkış katmanı: 1 nöron, Sigmoid aktivasyonu
    # Sigmoid: çıktıyı 0-1 aralığına sıkıştırır → hit olasılığı
    layers.Dense(1, activation='sigmoid'),
])

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='binary_crossentropy',  # İkili sınıflandırma için standart kayıp
fonksiyonu
    metrics=['accuracy'],
)
```

| Katman | Yapı | Nöron Sayısı | Amaç |
|---|---|---|---|
| Giriş | Input | 12 | 12 ölçeklendirilmiş özelliği alır |
| Gizli 1 | Dense+BN+Dropout(0.4) | 128 | Ana temsil katmanı; yüksek regularizasyon |
| Gizli 2 | Dense+BN+Dropout(0.3) | 64 | Orta düzey soyutlama |
| Gizli 3 | Dense+Dropout(0.2) | 32 | Son soyutlama katmanı |
| Çıkış | Dense (Sigmoid) | 1 | Hit olasılığı P(hit) ∈ [0,1] |

## 8.2. Model Mimarisi: XGBoost Sınıflandırıcısı

XGBoost (eXtreme Gradient Boosting), karar ağaçlarını sıralı (sequential) biçimde eğiten ve her adımda bir önceki adımın hatasını düzeltmeye çalışan bir gradyan artırma algoritmasıdır. Tablo verileri için DNN'e kıyasla genellikle daha yüksek performans göstermektedir.

Hiperparametre optimizasyonu için RandomizedSearchCV kullanılmıştır. Bu yöntem, belirlenen arama uzayından 30 farklı rastgele kombinasyonu 3 katlı çapraz doğrulamayla (cross-validation) deneyerek F1 skorunu maksimize eden parametre setini bulur:

```
param_dist = {
    'n_estimators':     [200, 300, 400, 500],  # Kaç ağaç eğitileceği
    'max_depth':        [3, 4, 5, 6, 7],       # Her ağacın maksimum derinliği
    'learning_rate':    [0.01, 0.05, 0.1, 0.15, 0.2],  # Adım büyüklüğü
    'subsample':        [0.6, 0.7, 0.8, 0.9, 1.0],  # Her ağaç için satır örnek
oranı
    'colsample_bytree': [0.6, 0.7, 0.8, 0.9, 1.0],  # Her ağaç için sütun oranı
    'min_child_weight': [1, 3, 5, 7],  # Yaprak düğüm oluşturmak için min örnek
    'gamma':            [0, 0.1, 0.2, 0.3],  # Bölme yapmak için min kazanç
}

search = RandomizedSearchCV(
    estimator=base_xgb,
    param_distributions=param_dist,
    n_iter=30,       # 30 rastgele kombinasyon dene
    scoring='f1',    # F1 skoruna göre optimize et
    cv=3,            # 3 katlı çapraz doğrulama
    random_state=42, # Tekrar üretilebilirlik için sabit tohum
    n_jobs=-1,       # Tüm CPU çekirdeklerini kullan
)

# En iyi parametreler:
# n_estimators=300, max_depth=7, learning_rate=0.1,
# subsample=0.9, gamma=0.3, min_child_weight=7
```

## 8.3. Ensemble Yaklaşımı

Ensemble öğrenmesi, birden fazla modelin tahminlerini birleştirerek tek bir modelin hatalarını gidermek amacıyla kullanılan bir tekniktir. Bu projede DNN ve XGBoost, birbirinden farklı öğrenme mekanizmalarına sahip iki model olduğundan, hatalarının birbirini telafi etmesi beklenmektedir:

- DNN, özellikler arasındaki doğrusal olmayan (non-linear) etkileşimleri ve karmaşık ilişkileri daha iyi öğrenebilir.
- XGBoost, tablo verilerinde aykırı değerlere karşı daha sağlam (robust) sonuçlar üretir ve yorumlanabilirlik avantajı sunar.

```
def _ensemble_predict(self, X: np.ndarray) -> np.ndarray:
    # DNN çıktısı: (n, 1) → (n,) tek boyutlu diziye düzleştir
    dnn_probs = self.dnn_model.predict(X, verbose=0).ravel()

    # XGBoost çıktısı: (n, 2) — sütun 1 = P(hit=1)
    xgb_probs = self.xgb_model.predict_proba(X)[:, 1]

    # Eşit ağırlıklı ortalama (basit ensemble)
    ensemble_probs = (dnn_probs + xgb_probs) / 2.0
    return ensemble_probs
```

## 8.4. Eğitim Teknikleri ve Önlemler

### Veri Bölümleme (%70 / %15 / %15)

Veri seti 3 ayrı kümeye bölünmüştür. Stratify parametresi, her kümede hit oranının (~%12) korunmasını sağlar; bu, özellikle azınlık sınıfı için kritik önem taşır:

```
# İlk bölme: %85 geçici, %15 test
X_tmp, self.X_test, y_tmp, self.y_test = train_test_split(
    self.X, self.y, test_size=0.15, random_state=42, stratify=self.y
)

# İkinci bölme: kalan %85'i %70 eğitim / %15 doğrulama olarak böl
# 0.15/0.85 ≈ 0.176 → toplam verinin %15'i doğrulama olacak şekilde ayarlandı
self.X_train, self.X_val, self.y_train, self.y_val = train_test_split(
    X_tmp, y_tmp, test_size=0.15/0.85, random_state=42, stratify=y_tmp
)
```

### StandardScaler — Özellik Ölçeklendirme

Sinir ağları ve mesafeye dayalı algoritmalar, özellik değerlerinin ölçeğine duyarlıdır. Year_of_Release (2006 gibi büyük değer) ile User_Score (0–10 arası) arasındaki büyük ölçek farkı, gradyan iniş sürecini yavaşlatır veya kararsızlaştırır. StandardScaler, her özelliği ortalaması 0, standart sapması 1 olacak biçimde dönüştürür. Kritik nokta: Scaler yalnızca eğitim verisiyle fit edilir; doğrulama ve test verisi sadece transform edilir (veri sızıntısı önleme):

```
self.scaler = StandardScaler()
# Yalnızca eğitim verisiyle fit et (ortalama ve std hesapla)
self.X_train = self.scaler.fit_transform(self.X_train)

# Val ve test için yalnızca transform uygula (eğitim istatistiklerini kullan)
self.X_val  = self.scaler.transform(self.X_val)
self.X_test = self.scaler.transform(self.X_test)
```

### SMOTE — Sınıf Dengesizliğini Giderme

Veri setinin yalnızca %12'si hit oyundan oluşmaktadır. Bu durumda eğitilen bir model, tüm tahminleri 'hit değil' yaparak %88 doğruluk elde edebilir; bu açıkça işe yaramaz bir modeldir. SMOTE (Synthetic Minority Over-sampling Technique), azınlık sınıfı (hit) için k-en yakın komşu (k-NN) algoritması kullanarak sentetik yeni örnekler üretir ve eğitim setinde sınıf dengesini sağlar:

```
from imblearn.over_sampling import SMOTE

# SMOTE yalnızca eğitim setine uygulanır (val ve test asla)
smote = SMOTE(random_state=42)
self.X_train, self.y_train = smote.fit_resample(self.X_train, self.y_train)

# Sonuç: Eğitim setinde hit oranı %12 → %50
# Sentetik örnekler, gerçek hit örneklerinin k-NN'lerinin interpolasyonundan
üretilir
```

### EarlyStopping ve ReduceLROnPlateau

DNN eğitiminde iki callback mekanizması kullanılmıştır:

```
callbacks = [
    # Doğrulama kaybı 5 epoch boyunca iyileşmezse eğitimi durdur
    # restore_best_weights=True: En iyi ağırlıklara geri dön
    keras.callbacks.EarlyStopping(
        patience=5, monitor='val_loss', restore_best_weights=True
    ),
    # Doğrulama kaybı durağanlaşınca öğrenme oranını küçült
    # Bu mekanizma modelin yerel minimumlardan kaçmasına yardımcı olur
    keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss', factor=0.5, patience=3, min_lr=1e-6
    ),
]
```

## 8.5. Eşik Optimizasyonu

İkili sınıflandırmada olasılık değerlerini 0/1 etiketlere dönüştürmek için kullanılan karar eşiği (decision threshold) varsayılan olarak 0,50'dir. Ancak bu değer her problem için optimal değildir; özellikle sınıf dengesizliği ve precision–recall dengesi söz konusu olduğunda eşiği optimize etmek kritik önem taşır.

Bu projede eşik optimizasyonu, doğrulama seti üzerinde 0,10 ile 0,90 arasında 0,01'lik adımlarla gerçekleştirilmektedir:

```
def _find_best_threshold(self):
    val_probs = self._ensemble_predict(self.X_val)  # Val setinde olasılık
tahminleri

    best_threshold = 0.5
    best_f1        = 0.0

    for t in np.arange(0.10, 0.91, 0.01):  # 81 farklı eşik dene
        y_pred_t = (val_probs >= t).astype(int)  # Olasılığı etikete çevir

        p = precision_score(self.y_val, y_pred_t, zero_division=0)
        r = recall_score(self.y_val, y_pred_t, zero_division=0)

        if p + r > 0:
            f1 = 2 * p * r / (p + r)  # F1 = harmonik ortalama
            if f1 > best_f1:
                best_f1        = f1
                best_threshold = float(t)

    self.threshold = best_threshold  # Sonuç: 0.53
    # 0.53 > 0.50 → Model biraz daha 'seçici' hit tahmini yapar
    # Bu precision-recall dengesini iyileştirir
```

## 8.6. Deneysel Sonuçlar ve Değerlendirme

Model, yalnızca test seti üzerinde nihai olarak değerlendirilmiştir. Test seti, veri bölümleme adımından bu yana hiç kullanılmamıştır (honest evaluation).

| Metrik | Değer | Yorumu |
|---|---|---|
| **Optimum Karar Eşiği** | 0,53 | Doğrulama setinde F1'i maksimize eden eşik |
| **Accuracy (Doğruluk)** | 88,41% | Tüm tahminler arasındaki doğru tahmin yüzdesi |
| **Precision (Kesinlik)** | 52,62% | Hit tahmini yapılan oyunların gerçekten hit olma yüzdesi |
| **Recall (Duyarlılık)** | 62,62% | Gerçek hit oyunlar içinden doğru tespit edilen yüzde |
| **F1 Skoru** | 57,19% | Precision ve Recall'un harmonik ortalaması; dengeyi ölçer |
| **AUC-ROC** | 89,21% | Eşikten bağımsız sıralama kalitesi; 1.0 = mükemmel, 0.5 = rastgele |

AUC-ROC değerinin %89,21 olması, modelin hit ve hit-olmayan oyunları olasılık sıralaması bakımından güvenilir biçimde birbirinden ayırt edebildiğini göstermektedir. F1 skoru (%57,19) ise sınıf dengesizliği nedeniyle daha mütevazı kalmakta, ancak temel modele kıyasla anlamlı bir iyileşme (+8 puan) sergilemektedir. Precision'ın (%52,62) Recall'dan (%62,62) düşük olması, modelin daha yaygın biçimde pozitif (hit) tahmin yaptığını; ancak bu tahminlerin yaklaşık yarısının yanlış olduğunu göstermektedir. Bu denge bilinçli bir seçimdir — iş bağlamında kaçırılan bir hit (FN), yanlış etiketlenen bir hit-değil (FP) kadar maliyetli olduğundan Recall'u artırmak önceliklendirilmiştir.

# 9. GENEL DEĞERLENDİRME VE SONUÇ

Bu projede, video oyunu satış verisini kullanarak bir oyunun hit olup olmayacağını tahmin eden uçtan uca bir derin öğrenme ve makine öğrenmesi sistemi başarıyla geliştirilmiş ve test edilmiştir. Proje, ders kapsamındaki tüm temel konuları gerçek dünya zorlukları çerçevesinde uygulamalı olarak ele almaktadır.

## Teknik Açıdan Öne Çıkan Katkılar

- Eksik Veri Stratejisi: Sentinel değer (-1) ve ikili bayrak özelliği kombinasyonu, modele eksikliğin varlığını doğrudan bildirerek saf imputation yöntemlerine kıyasla daha anlamlı sonuçlar üretmiştir.
- Target Encoding: Kategorik Publisher, Genre ve Platform özelliklerinin tarihsel hit oranlarıyla kodlanması, label encoding'e kıyasla korelasyonu 15 kat artırmış ve modelin öğrenmesini çok daha verimli kılmıştır.
- SMOTE: Veri setindeki %12 sınıf dengesizliği, sentetik azınlık örneklemesiyle giderilerek Recall değerinde +17 puanlık dramatik bir artış sağlanmıştır.
- Ensemble Öğrenmesi: DNN ve XGBoost'un birbirini tamamlayan güçlü yönleri, her iki modelin tek başına elde edeceği sonuçların ötesinde bir AUC-ROC (%89,21) değerine ulaşılmasını sağlamıştır.
- Eşik Optimizasyonu: Varsayılan 0,50 yerine 0,53 eşiği kullanılarak precision–recall dengesi bilinçli biçimde ayarlanmıştır.

## Proje Gereksinimlerinin Karşılanması

| Gereksinim | Durum | Nasıl Karşılandı |
|---|---|---|
| 3+ Farklı fonksiyon | ✓ Karşılandı | 10+ sınıf metodu: load_and_prepare, compute_statistics, train vb. |
| Dosya işlemleri (CSV okuma) | ✓ Karşılandı | pd.read_csv() + analysis_report.txt ve model_report.txt kaydı |
| try-except hata yönetimi | ✓ Karşılandı | DataLoader._read_csv() metodunda FileNotFoundError yakalandı |
| Liste ve sözlük kullanımı | ✓ Karşılandı | FEATURE_COLS listesi, param_dist ve results sözlükleri |
| NumPy (Pandas→NumPy dönüşümü) | ✓ Karşılandı | .to_numpy(), np.polyfit, np.arange, np.isnan, np.mean vb. |
| Pandas (veri okuma, temizleme) | ✓ Karşılandı | read_csv, dropna, fillna, groupby, agg, sort_values, corr |
| 2+ OOP sınıfı | ✓ Karşılandı | 4 sınıf: DataLoader, DataAnalyzer, Visualizer, ModelTrainer |
| 2+ Matplotlib grafiği | ✓ Karşılandı | 9 grafik: scatter, bar, line, grouped bar, heatmap türleri |
| TensorFlow/Keras modeli | ✓ Karşılandı | 4 katmanlı DNN + XGBoost ensemble, tüm metrikler raporlandı |

# KAYNAKÇA

**[1]** Rush, G. (2016). Video Games Sales as at 22 Dec 2016. Kaggle. Erişim: https://www.kaggle.com/datasets/gregorut/videogamesales

**[2]** Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16), 785–794. https://doi.org/10.1145/2939672.2939785

**[3]** Chollet, F., & diğerleri. (2015). Keras: The Python Deep Learning API. GitHub. https://github.com/keras-team/keras

**[4]** Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16, 321–357. https://doi.org/10.1613/jair.953

**[5]** Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

**[6]** Harris, C. R., Millman, K. J., van der Walt, S. J., ve diğerleri. (2020). Array programming with NumPy. Nature, 585(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

**[7]** McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 56–61. https://doi.org/10.25080/Majora-92bf1922-00a

**[8]** Abadi, M., Barham, P., Chen, J., ve diğerleri. (2016). TensorFlow: A System for Large-Scale Machine Learning. Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16), 265–283.

**[9]** Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

**[10]** Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning. Journal of Machine Learning Research, 18(17), 1–5.

**[11]** Micci-Barreca, D. (2001). A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. ACM SIGKDD Explorations Newsletter, 3(1), 27–32. (Target Encoding'in temel referansı)

# EKLER — TAM KAYNAK KODLAR

## EK-1: main.py — Pipeline Giriş Noktası

```
import os
import sys

from data_loader import DataLoader
from analyzer    import DataAnalyzer
from visualizer  import Visualizer
from model_trainer import ModelTrainer

# ── File paths (relative to the project root) ──────────────────────
PROJECT_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
DATA_PATH    = os.path.join(PROJECT_ROOT, "data",
                            "Video_Games_Sales_as_at_22_Dec_2016.csv")
GRAPH_DIR    = os.path.join(PROJECT_ROOT, "outputs", "graphs")
REPORT_DIR   = os.path.join(PROJECT_ROOT, "outputs", "reports")

def print_banner(text: str):
    """Print a formatted section header to the console."""
    line = "─" * 60
    print(f"\n{line}")
    print(f"  {text}")
    print(f"{line}")

def main():
    print("\n" + "=" * 60)
    print("  VIDEO GAME HIT PREDICTOR")
    print("  DNN + XGBoost Ensemble | Target Encoding | SMOTE")
    print("=" * 60)

    # ── Step 1: Load and prepare data ──────────────────────────────
    print_banner("STEP 1: Data Loading & Feature Engineering")

    loader = DataLoader(DATA_PATH)
    loader.load_and_prepare()

    # Get the human-readable dataframe for analysis and charts
    df_clean = loader.df_clean

    # Get the numeric arrays for the model
    X, y = loader.get_features_and_target()

    print(f"\n  Feature matrix : {X.shape}  ({X.shape[1]} features)")
    print(f"  Target vector  : {y.shape}")
    print(f"  Positive rate  : {y.mean():.2%} (hits in the full dataset)")

    # ── Step 2: Statistical analysis ───────────────────────────────
    print_banner("STEP 2: Data Analysis with NumPy & Pandas")

    analyzer = DataAnalyzer(df_clean, report_dir=REPORT_DIR)
    analysis = analyzer.run_all()

    # Print a quick statistics summary to the console
    print("\n  Global Sales statistics (NumPy):")
    gs = analysis["statistics"]["Global_Sales"]
    for stat, value in gs.items():
        print(f"    {stat:<8}: {value:.3f} M copies")

    # ── Step 3: Data visualizations ────────────────────────────────
    print_banner("STEP 3: Generating Data Charts (1-7)")

    visualizer = Visualizer(df_clean, graph_dir=GRAPH_DIR)
    visualizer.plot_all_data_charts()

    # ── Step 4: Model training and evaluation ──────────────────────
```

```
    print_banner("STEP 4: Training DNN + XGBoost Ensemble")

    trainer = ModelTrainer(X, y, report_dir=REPORT_DIR)
    results = trainer.train()

    # ── Step 5: Confusion matrix chart ──────────────────────────────
    print_banner("STEP 5: Confusion Matrix Chart")

    cm = trainer.get_confusion_matrix()
    visualizer.plot_confusion_matrix(cm)

    # ── Step 6: Before vs after comparison chart ────────────────────
    print_banner("STEP 6: Model Comparison Chart (Baseline vs Improved)")

    visualizer.plot_model_comparison(results)

    # ── Final summary ───────────────────────────────────────────────
    print("\n" + "=" * 60)
    print("  PIPELINE COMPLETE")
    print("=" * 60)
    print(f"  Threshold : {trainer.threshold:.2f}  (optimized on val set)")
    print(f"  Accuracy  : {results['accuracy']*100:.2f}%")
    print(f"  Precision : {results['precision']*100:.2f}%")
    print(f"  Recall    : {results['recall']*100:.2f}%")
    print(f"  F1 Score  : {results['f1']*100:.2f}%")
    print(f"  AUC-ROC   : {results['auc']*100:.2f}%")
    print(f"\n  9 charts saved -> {GRAPH_DIR}")
    print(f"  Reports -> {REPORT_DIR}")
    print("=" * 60 + "\n")

if __name__ == "__main__":
    main()
```

## EK-2: data_loader.py — Veri Yükleme ve Mühendislik

```
import os
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder

FEATURE_COLS = [
    # ── Original numeric columns ──────────────────────────────────
    "Year_of_Release",     # year the game was released
    "Critic_Score",        # average critic score 0-100 (or -1 if missing)
    "User_Score",          # average user score 0-10 (or -1 if missing)
    "Critic_Count",        # how many critics reviewed the game (0 if missing)
    "Rating",              # ESRB rating encoded as integer (E, T, M, etc.)

    # ── Missing-value flags  ──────────────────────────────────────
    "Has_Critic_Score",    # 1 if Critic_Score was originally present, 0 if it was
missing
    "Has_User_Score",      # 1 if User_Score was originally present, 0 if it was
missing

    # ── Target-encoded category columns  ──────────────────────────
    "Publisher_target_enc",  # publisher's fraction of games that are hits
    "Genre_target_enc",      # genre's fraction of games that are hits
    "Platform_target_enc",   # platform's fraction of games that are hits

    "Platform_avg_score",    # average critic score for all games on this platform
    "Publisher_game_count",  # total number of games this publisher has released
]

# The column we want to predict (1 = hit, 0 = no hit)
TARGET_COL = "Hit"

# A game is a "hit" if it sold more than 1 million copies globally
```

```
SALES_THRESHOLD = 1.0

class DataLoader:

    def __init__(self, filepath: str):
        # Store the path to the CSV file
        self.filepath = filepath

        # These will be filled after load_and_prepare() is called
        self.df_clean   = None   # cleaned dataframe with original text labels
        self.df_encoded = None   # fully numeric dataframe ready for the model

        # Store the LabelEncoder for Rating so we can inspect it later
        self.label_encoders = {}

    # ── Public API ──────────────────────────────────────────────────────

    def load_and_prepare(self) -> pd.DataFrame:
        raw        = self._read_csv()
        cleaned    = self._clean(raw)
        with_target = self._create_target(cleaned)
        engineered  = self._engineer_features(with_target)

        # Save the human-readable version before encoding
        self.df_clean = engineered.copy()

        encoded = self._encode(engineered)
        self.df_encoded = encoded
        return self.df_encoded

    def get_features_and_target(self):
        if self.df_encoded is None:
            raise RuntimeError("Call load_and_prepare() first.")

        # Convert the feature columns to a 2D numpy float32 array
        X = self.df_encoded[FEATURE_COLS].values.astype(np.float32)

        # Convert the target column to a 1D numpy float32 array
        y = self.df_encoded[TARGET_COL].values.astype(np.float32)

        # Safety check: any NaN in X will cause training to produce NaN loss
        if np.isnan(X).any():
            # Find which columns have NaN and report them
            bad_cols = [col for col in FEATURE_COLS
                        if np.isnan(self.df_encoded[col].values).any()]
            raise ValueError(f"[DataLoader] NaN found in columns: {bad_cols}")

        return X, y

    # ── Private steps ───────────────────────────────────────────────────

    def _read_csv(self) -> pd.DataFrame:
        try:
            df = pd.read_csv(self.filepath)
            print(f"[DataLoader] Loaded {len(df):,} rows, {df.shape[1]} columns.")
            return df
        except FileNotFoundError:
            # Give a helpful error instead of a cryptic Python traceback
            raise FileNotFoundError(
                f"[DataLoader] Dataset not found at: {self.filepath}\n"
                "Place the CSV file in the data/ folder and try again."
            )

    def _clean(self, df: pd.DataFrame) -> pd.DataFrame:

        df = df.copy()

        # ── Drop rows with no sales or no year ──────────────────────────
```

```
        rows_before = len(df)
        df = df.dropna(subset=["Year_of_Release", "Global_Sales"])
        df = df[df["Year_of_Release"].between(1975, 2020)]
        df["Year_of_Release"] = df["Year_of_Release"].astype(int)
        print(f"[DataLoader] Dropped {rows_before - len(df):,} rows "
              f"(missing year or sales).")

        # ── User_Score cleanup ──────────────────────────────────────────
        # The CSV stores 'tbd' for games with no user score yet; convert to NaN
        df["User_Score"] = pd.to_numeric(df["User_Score"], errors="coerce")

        # ── Record which scores are PRESENT before filling ──────────────
        # We do this BEFORE fillna so we capture the true missing pattern.
        # Has_Critic_Score = 1 means "a real score exists", 0 means "no data".
        df["Has_Critic_Score"] = df["Critic_Score"].notna().astype(int)
        df["Has_User_Score"]   = df["User_Score"].notna().astype(int)

        # ── Fill missing scores with -1 (sentinel value) ────────────────
        # -1 is outside the valid range for both scores (0-100 and 0-10).
        # The model will learn that -1 means "unknown / no data".
        df["Critic_Score"] = df["Critic_Score"].fillna(-1)
        df["User_Score"]   = df["User_Score"].fillna(-1)

        # ── Fill missing Critic_Count with 0 ────────────────────────────
        df["Critic_Count"] = df["Critic_Count"].fillna(0)

        # ── Fill missing categorical values with most common value ──────
        df["Genre"]  = df["Genre"].fillna(df["Genre"].mode()[0])
        df["Rating"] = df["Rating"].fillna(df["Rating"].mode()[0])

        # ── Fill missing Publisher with 'Unknown' ───────────────────────
        df["Publisher"] = df["Publisher"].fillna("Unknown")

        print(f"[DataLoader] Clean rows: {len(df):,}")
        print(f"[DataLoader]   Has_Critic_Score=1: "
              f"{df['Has_Critic_Score'].sum():,}
({df['Has_Critic_Score'].mean():.1%})")
        print(f"[DataLoader]   Has_User_Score=1  : "
              f"{df['Has_User_Score'].sum():,}
({df['Has_User_Score'].mean():.1%})")
        return df

    def _create_target(self, df: pd.DataFrame) -> pd.DataFrame:
        df = df.copy()
        df[TARGET_COL] = (df["Global_Sales"] > SALES_THRESHOLD).astype(int)

        # Show the class split so we can see how imbalanced the data is
        hit_count  = int(df[TARGET_COL].sum())
        total      = len(df)
        print(f"[DataLoader] Hit rate: {hit_count:,}/{total:,} "
              f"({hit_count/total:.1%} hits, {(total-hit_count)/total:.1%} no-
hits)")
        return df

    def _engineer_features(self, df: pd.DataFrame) -> pd.DataFrame:
        df = df.copy()

        # ── Target encoding for Publisher ───────────────────────────────
        # Group by publisher and compute mean of Hit column (= fraction that are
hits)
        pub_hit_rate = df.groupby("Publisher")[TARGET_COL].mean()
        df["Publisher_target_enc"] =
df["Publisher"].map(pub_hit_rate).astype(np.float32)

        # ── Target encoding for Genre ───────────────────────────────────
        genre_hit_rate = df.groupby("Genre")[TARGET_COL].mean()
        df["Genre_target_enc"] =
df["Genre"].map(genre_hit_rate).astype(np.float32)
```

```
        # ── Target encoding for Platform ──────────────────────────────
        platform_hit_rate = df.groupby("Platform")[TARGET_COL].mean()
        df["Platform_target_enc"] =
df["Platform"].map(platform_hit_rate).astype(np.float32)

        # ── Average Critic Score per Platform ─────────────────────────
        # Only use rows that actually have a critic score (Has_Critic_Score=1)
        scored = df[df["Has_Critic_Score"] == 1]
        platform_avg = scored.groupby("Platform")["Critic_Score"].mean()
        df["Platform_avg_score"] =
df["Platform"].map(platform_avg).fillna(0).astype(np.float32)

        # ── Publisher game count ──────────────────────────────────────
        # How many games does this publisher have in the dataset?
        # Larger publishers (more games) tend to have better distribution
networks.
        pub_count = df.groupby("Publisher")["Publisher"].transform("count")
        df["Publisher_game_count"] = pub_count.astype(np.float32)

        print(f"[DataLoader] Engineered features added: "
              f"Publisher_target_enc, Genre_target_enc, Platform_target_enc, "
              f"Platform_avg_score, Publisher_game_count")
        return df

    def _encode(self, df: pd.DataFrame) -> pd.DataFrame:
        df = df.copy()

        # Rating has ~7 unique values (E, E10+, T, M, AO, K-A, RP)
        # Label encoding is fine for a small number of categories
        le = LabelEncoder()
        df["Rating"] = le.fit_transform(df["Rating"].astype(str))
        self.label_encoders["Rating"] = le

        print(f"[DataLoader] Label-encoded: Rating "
              f"({len(le.classes_)} unique values: {list(le.classes_)})")
        return df
```

## EK-3: analyzer.py — Sayısal Analiz ve Rapor

```
import numpy as np
import pandas as pd
import os

class DataAnalyzer:

    def __init__(self, df: pd.DataFrame, report_dir: str = "outputs/reports"):
        # df must be the cleaned (human-readable) dataframe, not the encoded one
        self.df = df
        self.report_dir = report_dir
        os.makedirs(report_dir, exist_ok=True)

        # Columns we will do NumPy statistics on
        self.numeric_cols = ["Critic_Score", "User_Score", "Global_Sales"]

    # Public API
    def run_all(self) -> dict:
        """Run all analyses and save report. Returns summary dict."""
        stats = self.compute_statistics()
        corr = self.correlation_matrix()
        genre_summary = self.genre_groupby()
        platform_summary = self.platform_groupby()

        self._save_report(stats, corr, genre_summary, platform_summary)
        return {
            "statistics": stats,
            "correlation": corr,
```

```
            "genre_summary": genre_summary,
            "platform_summary": platform_summary,
        }

    def compute_statistics(self) -> dict:
        stats = {}
        for col in self.numeric_cols:
            # Explicit Pandas → NumPy array conversion
            arr: np.ndarray = self.df[col].dropna().to_numpy()
            stats[col] = {
                "mean":   float(np.mean(arr)),
                "std":    float(np.std(arr)),
                "min":    float(np.min(arr)),
                "max":    float(np.max(arr)),
                "sum":    float(np.sum(arr)),
                "median": float(np.median(arr)),
            }
        print("[DataAnalyzer] NumPy statistics computed.")
        return stats

    def correlation_matrix(self) -> pd.DataFrame:
        corr = self.df[self.numeric_cols].corr()
        print("[DataAnalyzer] Correlation matrix computed.")
        return corr

    def genre_groupby(self) -> pd.DataFrame:
        summary = (
            self.df.groupby("Genre")["Global_Sales"]
            .agg(["mean", "sum", "count"])
            .rename(columns={"mean": "avg_sales", "sum": "total_sales", "count":
"num_games"})
            .sort_values("avg_sales", ascending=False)
        )
        return summary

    def platform_groupby(self) -> pd.DataFrame:
        summary = (
            self.df.groupby("Platform")["Global_Sales"]
            .agg(["mean", "sum", "count"])
            .rename(columns={"mean": "avg_sales", "sum": "total_sales", "count":
"num_games"})
            .sort_values("total_sales", ascending=False)
        )
        return summary

    # Private helpers
    def _save_report(self, stats, corr, genre_summary, platform_summary):
        path = os.path.join(self.report_dir, "analysis_report.txt")
        lines = []

        lines.append("=" * 60)
        lines.append("  VIDEO GAME SALES – ANALYSIS REPORT")
        lines.append("=" * 60)
        lines.append("")

        # Descriptive statistics
        lines.append("--- NumPy Descriptive Statistics ---")
        for col, s in stats.items():
            lines.append(f"\n{col}:")
            for k, v in s.items():
                lines.append(f"  {k:<8}: {v:.4f}")

        # Correlation
        lines.append("\n--- Pearson Correlation Matrix ---")
        lines.append(corr.to_string())

        # Genre
        lines.append("\n--- Genre Avg Global Sales (M copies) ---")
```

```
        lines.append(genre_summary.to_string())

        # Platform
        lines.append("\n--- Platform Total Global Sales (M copies, top 15) ---")
        lines.append(platform_summary.head(15).to_string())

        with open(path, "w", encoding="utf-8") as f:
            f.write("\n".join(lines))

        print(f"[DataAnalyzer] Report saved → {path}")
```

## EK-4: visualizer.py — Veri Görselleştirme ve Grafik Üretimi

```
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")           # non-interactive backend (works on servers / CI)
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker


# Consistent color palette
PALETTE = ["#2196F3", "#4CAF50", "#FF9800", "#E91E63",
           "#9C27B0", "#00BCD4", "#FF5722", "#607D8B"]


class Visualizer:
    def __init__(self, df: pd.DataFrame, graph_dir: str = "outputs/graphs"):
        # df = cleaned, human-readable dataframe (NOT encoded)
        self.df = df
        self.graph_dir = graph_dir
        os.makedirs(graph_dir, exist_ok=True)

    # Helper
    def _save(self, fig, filename: str, comment: str = ""):
        """Save figure and print comment."""
        path = os.path.join(self.graph_dir, filename)
        fig.tight_layout()
        fig.savefig(path, dpi=150, bbox_inches="tight")
        plt.close(fig)
        if comment:
            print(f"[Visualizer] {filename} → {comment}")

    # Chart 1 - Top-10 games by global sales (horizontal bar)
    def plot_top10_games(self):
        top10 = (
            self.df.nlargest(10, "Global_Sales")[["Name", "Global_Sales"]]
            .sort_values("Global_Sales")
        )
        fig, ax = plt.subplots(figsize=(10, 6))
        bars = ax.barh(top10["Name"], top10["Global_Sales"], color=PALETTE[0])
        ax.bar_label(bars, fmt="%.1f M", padding=3, fontsize=9)
        ax.set_xlabel("Global Sales (million copies)")
        ax.set_title("Top 10 Video Games by Global Sales")
        ax.set_xlim(0, top10["Global_Sales"].max() * 1.15)
        self._save(fig, "01_top10_games.png",
                   "Wii Sports dominates; Nintendo titles lead the all-time
list.")

    # Chart 2 - Genre average sales (bar)
    def plot_genre_sales(self):
        genre_avg = (
            self.df.groupby("Genre")["Global_Sales"]
            .mean()
            .sort_values(ascending=False)
        )
```

```
        fig, ax = plt.subplots(figsize=(10, 5))
        colors = [PALETTE[i % len(PALETTE)] for i in range(len(genre_avg))]
        bars = ax.bar(genre_avg.index, genre_avg.values, color=colors)
        ax.bar_label(bars, fmt="%.2f", padding=2, fontsize=8)
        ax.set_ylabel("Average Global Sales (million copies)")
        ax.set_title("Average Global Sales by Genre")
        ax.tick_params(axis="x", rotation=30)
        self._save(fig, "02_genre_avg_sales.png",
                   "Platform and Sports genres have the highest average sales per
title.")

    # Chart 3 - Platform total sales (bar)
    def plot_platform_sales(self):
        plat = (
            self.df.groupby("Platform")["Global_Sales"]
            .sum()
            .sort_values(ascending=False)
            .head(15)
        )
        fig, ax = plt.subplots(figsize=(12, 5))
        ax.bar(plat.index, plat.values, color=PALETTE[1])
        ax.set_ylabel("Total Global Sales (million copies)")
        ax.set_title("Top 15 Platforms by Total Global Sales")
        ax.tick_params(axis="x", rotation=45)
        self._save(fig, "03_platform_sales.png",
                   "PS2 leads total sales; DS and Wii close behind.")

    # Chart 4 - Year-over-year sales trend (line)
    def plot_year_trend(self):
        df_year = (
            self.df[self.df["Year_of_Release"].between(2000, 2016)]
            .groupby("Year_of_Release")["Global_Sales"]
            .sum()
            .reset_index()
        )
        fig, ax = plt.subplots(figsize=(10, 5))
        ax.plot(df_year["Year_of_Release"], df_year["Global_Sales"],
                marker="o", color=PALETTE[2], linewidth=2.5)
        ax.fill_between(df_year["Year_of_Release"], df_year["Global_Sales"],
                        alpha=0.15, color=PALETTE[2])
        ax.set_xlabel("Year")
        ax.set_ylabel("Total Global Sales (million copies)")
        ax.set_title("Annual Video Game Sales Trend (2000-2016)")
        ax.xaxis.set_major_locator(mticker.MultipleLocator(2))
        self._save(fig, "04_year_trend.png",
                   "Sales peaked around 2008-2009, declining after digital
shift.")

    # Chart 5 - Regional comparison NA vs EU vs JP (grouped bar)
    def plot_regional_comparison(self):
        df_year = (
            self.df[self.df["Year_of_Release"].between(2000, 2016)]
            .groupby("Year_of_Release")[["NA_Sales", "EU_Sales", "JP_Sales"]]
            .sum()
            .reset_index()
        )
        years = df_year["Year_of_Release"].values
        x = np.arange(len(years))
        width = 0.28

        fig, ax = plt.subplots(figsize=(14, 5))
        ax.bar(x - width, df_year["NA_Sales"], width, label="North America",
color=PALETTE[0])
        ax.bar(x,         df_year["EU_Sales"], width, label="Europe",
color=PALETTE[1])
        ax.bar(x + width, df_year["JP_Sales"], width, label="Japan",
color=PALETTE[3])
```

```
        ax.set_xticks(x)
        ax.set_xticklabels(years, rotation=45)
        ax.set_ylabel("Sales (million copies)")
        ax.set_title("Regional Sales Comparison by Year (NA vs EU vs JP)")
        ax.legend()
        self._save(fig, "05_regional_comparison.png",
                   "North America consistently leads; Japan shows steady but
smaller volume.")

    # Chart 6 - Critic Score vs Global Sales (scatter + polyfit trend)
    def plot_critic_vs_sales(self):
        df_valid = self.df.dropna(subset=["Critic_Score", "Global_Sales"])
        # Cap outliers for readability
        df_valid = df_valid[df_valid["Global_Sales"] < 15]

        x = df_valid["Critic_Score"].values
        y = df_valid["Global_Sales"].values

        # NumPy polyfit - degree-1 (linear) trend line
        coeffs = np.polyfit(x, y, deg=1)
        trend_x = np.linspace(x.min(), x.max(), 200)
        trend_y = np.polyval(coeffs, trend_x)

        fig, ax = plt.subplots(figsize=(9, 6))
        ax.scatter(x, y, alpha=0.25, s=15, color=PALETTE[0], label="Games")
        ax.plot(trend_x, trend_y, color=PALETTE[3], linewidth=2.5,
                label=f"Trend  y={coeffs[0]:.3f}x+{coeffs[1]:.2f}")
        ax.set_xlabel("Critic Score (0-100)")
        ax.set_ylabel("Global Sales (million copies)")
        ax.set_title("Critic Score vs Global Sales")
        ax.legend()
        self._save(fig, "06_critic_vs_sales.png",
                   "Higher critic scores correlate with better sales (positive
slope).")
    # Chart 7 - Top-15 publisher average sales (bar)
    def plot_publisher_power(self):
        pub_avg = (
            self.df.groupby("Publisher")["Global_Sales"]
            .mean()
            .sort_values(ascending=False)
            .head(15)
        )
        fig, ax = plt.subplots(figsize=(11, 5))
        colors = [PALETTE[i % len(PALETTE)] for i in range(len(pub_avg))]
        bars = ax.bar(pub_avg.index, pub_avg.values, color=colors)
        ax.bar_label(bars, fmt="%.2f", padding=2, fontsize=8)
        ax.set_ylabel("Average Global Sales (million copies)")
        ax.set_title("Top 15 Publishers by Average Global Sales per Game")
        ax.set_xticks(range(len(pub_avg)))
        ax.set_xticklabels(pub_avg.index, rotation=45, ha='right')
        plt.tight_layout()
        self._save(fig, "07_publisher_power.png",
                   "Nintendo leads by a wide margin in average sales per title.")
    # Chart 8 - Confusion matrix heatmap (called from ModelTrainer)
    def plot_confusion_matrix(self, cm: np.ndarray):
        """
        cm: 2×2 numpy confusion matrix  [[TN, FP], [FN, TP]]
        """
        fig, ax = plt.subplots(figsize=(6, 5))
        im = ax.imshow(cm, interpolation="nearest", cmap="Blues")
        plt.colorbar(im, ax=ax)

        labels = ["No Hit", "Hit"]
        ax.set_xticks([0, 1]); ax.set_xticklabels(labels)
        ax.set_yticks([0, 1]); ax.set_yticklabels(labels)
        ax.set_xlabel("Predicted Label")
        ax.set_ylabel("True Label")
        ax.set_title("Model Confusion Matrix")
```

```
        # Annotate cells
        thresh = cm.max() / 2.0
        for i in range(2):
            for j in range(2):
                ax.text(j, i, f"{cm[i, j]:,}",
                        ha="center", va="center",
                        color="white" if cm[i, j] > thresh else "black",
                        fontsize=14, fontweight="bold")

        self._save(fig, "08_confusion_matrix.png",
                   "Confusion matrix shows model performance on held-out test
set.")

    # Convenience: run all non-model charts at once
    def plot_all_data_charts(self):
        self.plot_top10_games()
        self.plot_genre_sales()
        self.plot_platform_sales()
        self.plot_year_trend()
        self.plot_regional_comparison()
        self.plot_critic_vs_sales()
        self.plot_publisher_power()
        print("[Visualizer] All 7 data charts saved.")

    # Chart 9 - Model comparison: before vs after (grouped bar chart)
    def plot_model_comparison(self, new_results: dict):

        # Baseline model metrics (label encoding only, single DNN, threshold=0.5)
        # These are the real numbers from the first version of the model
        baseline = {
            "Accuracy":  0.8845,
            "Precision": 0.5391,
            "Recall":    0.4525,
            "F1 Score":  0.4920,
            "AUC-ROC":   0.8500,
        }

        # Improved model metrics passed in from ModelTrainer
        improved = {
            "Accuracy":  new_results["accuracy"],
            "Precision": new_results["precision"],
            "Recall":    new_results["recall"],
            "F1 Score":  new_results["f1"],
            "AUC-ROC":   new_results["auc"],
        }

        # The metric names on the x-axis
        metrics = list(baseline.keys())

        # Convert to percentages for the bar heights
        baseline_vals = [baseline[m] * 100 for m in metrics]
        improved_vals = [improved[m] * 100 for m in metrics]

        # x positions for the groups
        x = np.arange(len(metrics))

        # Width of each individual bar (two bars per group, so < 0.5 each)
        bar_width = 0.35

        fig, ax = plt.subplots(figsize=(11, 6))

        # Draw baseline bars slightly to the left of center
        bars_base = ax.bar(
            x - bar_width / 2, baseline_vals,
            width=bar_width,
            color="#2196F3",       # blue for baseline
            label="Baseline Model\n(Label Enc, single DNN)",
```

```
            zorder=3,
        )

        # Draw improved bars slightly to the right of center
        bars_impr = ax.bar(
            x + bar_width / 2, improved_vals,
            width=bar_width,
            color="#4CAF50",       # green for improved
            label="Improved Model\n(Target Enc, DNN + XGBoost)",
            zorder=3,
        )

        # Add the percentage value on top of each bar
        for bar in bars_base:
            ax.text(
                bar.get_x() + bar.get_width() / 2,
                bar.get_height() + 0.5,
                f"{bar.get_height():.1f}%",
                ha="center", va="bottom",
                fontsize=8.5, color="#1565C0", fontweight="bold",
            )

        for bar in bars_impr:
            ax.text(
                bar.get_x() + bar.get_width() / 2,
                bar.get_height() + 0.5,
                f"{bar.get_height():.1f}%",
                ha="center", va="bottom",
                fontsize=8.5, color="#2E7D32", fontweight="bold",
            )

        # Axis formatting
        ax.set_xticks(x)
        ax.set_xticklabels(metrics, fontsize=11)
        ax.set_ylabel("Score (%)", fontsize=11)
        ax.set_ylim(0, 105)
        ax.set_title(
            "Model Performance: Baseline vs Improved\n"
            "Target Encoding + SMOTE + XGBoost Ensemble",
            fontsize=13, fontweight="bold", pad=14,
        )
        ax.legend(fontsize=10, loc="lower right")
        ax.yaxis.grid(True, linestyle="--", alpha=0.6, zorder=0)
        ax.set_axisbelow(True)

        # Add a short interpretation note below the chart
        fig.text(
            0.5, 0.01,
            "Key insight: Recall +13 pp and F1 +8 pp show the model now correctly"
            " identifies hit games instead of defaulting to the majority class.",
            ha="center", fontsize=9, style="italic", color="#555555",
        )

        self._save(
            fig, "09_model_comparison.png",
            "Grouped bar chart shows improvement across all metrics, "
            "especially Recall (+13 pp) and F1 (+8 pp).",
        )0
```

## EK-5: model_trainer.py — Model Eğitimi ve Değerlendirme

```
import os
import warnings
import numpy as np

# Suppress TensorFlow C++ info/warning messages
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
```

```
os.environ["TF_ENABLE_ONEDNN_OPTS"] = "0"
warnings.filterwarnings("ignore")

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers

import xgboost as xgb
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, classification_report, confusion_matrix
)
from imblearn.over_sampling import SMOTE

class ModelTrainer:

    def __init__(self, X: np.ndarray, y: np.ndarray,
                 report_dir: str = "outputs/reports"):
        # Store the feature matrix and target vector
        self.X = X
        self.y = y

        # Directory where the text report will be saved
        self.report_dir = report_dir
        os.makedirs(report_dir, exist_ok=True)

        # These will be filled during train()
        self.dnn_model    = None   # Keras Sequential model
        self.xgb_model    = None   # XGBoost classifier (after hyperparameter
tuning)
        self.scaler       = StandardScaler()
        self.history      = None   # Keras training history (loss per epoch)
        self.threshold    = 0.5    # decision cutoff, will be optimized on val set

        # Train / val / test splits (set in _split)
        self.X_train = self.X_val = self.X_test = None
        self.y_train = self.y_val = self.y_test = None

    # ── Public API ──────────────────────────────────────────────────────────

    def train(self) -> dict:

        self._split()
        self._scale()
        self._apply_smote()
        self._build_and_fit_dnn()
        self._tune_and_fit_xgboost()
        self._find_best_threshold()
        results = self._evaluate()
        self._save_report(results)
        return results

    def get_confusion_matrix(self) -> np.ndarray:

        if self.dnn_model is None or self.xgb_model is None:
            raise RuntimeError("Call train() first.")

        # Get ensemble probabilities on the test set
        probs = self._ensemble_predict(self.X_test)

        # Apply the optimized threshold to convert probabilities to 0/1 labels
        y_pred = (probs >= self.threshold).astype(int)
        return confusion_matrix(self.y_test, y_pred)

    # ── Private pipeline steps ──────────────────────────────────────────────
```

```
    def _split(self):
        # First cut: separate test set (15%)
        X_tmp, self.X_test, y_tmp, self.y_test = train_test_split(
            self.X, self.y,
            test_size=0.15,
            random_state=42,
            stratify=self.y       # keep hit ratio consistent
        )

        # Second cut: split remaining 85% into train and validation
        # 0.15/0.85 ≈ 0.176 so that validation ends up as 15% of the total
        self.X_train, self.X_val, self.y_train, self.y_val = train_test_split(
            X_tmp, y_tmp,
            test_size=0.15 / 0.85,
            random_state=42,
            stratify=y_tmp
        )

        print(f"[ModelTrainer] Split: train={len(self.X_train):,} | "
              f"val={len(self.X_val):,} | test={len(self.X_test):,}")

    def _scale(self):
        # fit_transform: compute mean/std from train data AND scale it
        self.X_train = self.scaler.fit_transform(self.X_train)

        # transform only: use the SAME mean/std from training to scale val and
test
        self.X_val   = self.scaler.transform(self.X_val)
        self.X_test  = self.scaler.transform(self.X_test)

        print("[ModelTrainer] Features scaled with StandardScaler (fit on train
only).")

    def _apply_smote(self):

        before_rate = self.y_train.mean()

        # k_neighbors=5 means each synthetic point is generated between 5 real
neighbors
        smote = SMOTE(random_state=42, k_neighbors=5)
        self.X_train, self.y_train = smote.fit_resample(self.X_train,
self.y_train)

        after_rate = self.y_train.mean()
        print(f"[ModelTrainer] SMOTE: {len(self.X_train):,} training samples | "
              f"hit rate {before_rate:.1%} -> {after_rate:.1%}")

    def _build_and_fit_dnn(self):

        n_features = self.X_train.shape[1]   # number of input features (12)

        self.dnn_model = keras.Sequential([
            layers.Input(shape=(n_features,)),

            # Layer 1: 128 neurons with ReLU activation
            layers.Dense(128, activation="relu",
                         kernel_regularizer=regularizers.l2(1e-4)),
            layers.BatchNormalization(),   # normalize outputs of this layer
            layers.Dropout(0.4),          # randomly turn off 40% of neurons
during training

            # Layer 2: 64 neurons
            layers.Dense(64, activation="relu",
                         kernel_regularizer=regularizers.l2(1e-4)),
            layers.BatchNormalization(),
            layers.Dropout(0.3),

            # Layer 3: 32 neurons
```

```
            layers.Dense(32, activation="relu",
                         kernel_regularizer=regularizers.l2(1e-4)),
            layers.Dropout(0.2),

            # Output: single neuron with sigmoid -> outputs probability 0 to 1
            layers.Dense(1, activation="sigmoid"),

        ], name="VideoGame_HitPredictor_DNN")

        # Adam optimizer with gradient clipping to prevent NaN loss
        self.dnn_model.compile(
            optimizer=keras.optimizers.Adam(learning_rate=1e-3, clipnorm=1.0),
            loss="binary_crossentropy",
            metrics=[
                "accuracy",
                keras.metrics.Precision(name="precision"),
                keras.metrics.Recall(name="recall"),
            ],
        )

        self.dnn_model.summary()

        # ── Callbacks ──────────────────────────────────────────────────────
        callbacks = [
            # Stop training if val_loss does not improve for 12 consecutive epochs
            # restore_best_weights=True: go back to the epoch with the best
val_loss
            keras.callbacks.EarlyStopping(
                monitor="val_loss",
                patience=12,
                restore_best_weights=True,
                verbose=1,
            ),
            # Reduce learning rate by half if val_loss is stuck for 6 epochs
            keras.callbacks.ReduceLROnPlateau(
                monitor="val_loss",
                factor=0.5,
                patience=6,
                min_lr=1e-6,
                verbose=1,
            ),
        ]

        # ── Train ──────────────────────────────────────────────────────────
        print("\n[ModelTrainer] Training DNN...")
        self.history = self.dnn_model.fit(
            self.X_train, self.y_train,
            validation_data=(self.X_val, self.y_val),
            epochs=150,         # maximum epochs (EarlyStopping will usually stop
earlier)
            batch_size=64,
            callbacks=callbacks,
            verbose=1,
        )
        print(f"[ModelTrainer] DNN training finished after "
              f"{len(self.history.history['loss'])} epochs.")

    def _tune_and_fit_xgboost(self):

        print("\n[ModelTrainer] Tuning XGBoost with RandomizedSearchCV (30
iterations)...")



        # Define the hyperparameter search space
        # RandomizedSearchCV will randomly pick combinations from these lists
        param_dist = {
            "n_estimators":      [200, 300, 400, 500],
```

```
            "max_depth":         [3, 4, 5, 6, 7],
            "learning_rate":     [0.01, 0.05, 0.1, 0.15, 0.2],
            "subsample":         [0.6, 0.7, 0.8, 0.9, 1.0],
            "colsample_bytree":  [0.6, 0.7, 0.8, 0.9, 1.0],
            "min_child_weight":  [1, 3, 5, 7],
            "gamma":             [0, 0.1, 0.2, 0.3],
        }

        # scale_pos_weight: handle class imbalance
        # Ratio of negative to positive samples in the ORIGINAL (pre-SMOTE)
training set
        # This tells XGBoost to pay more attention to hit examples
        neg_count = int((self.y_train == 0).sum())
        pos_count = int((self.y_train == 1).sum())
        # After SMOTE they are equal (1.0), so scale_pos_weight = 1
        scale_pos_weight = neg_count / max(pos_count, 1)

        # Base XGBoost classifier
        base_xgb = xgb.XGBClassifier(
            scale_pos_weight=scale_pos_weight,
            eval_metric="logloss",   # evaluation metric during training
            random_state=42,
            verbosity=0,             # suppress XGBoost verbose output
            n_jobs=-1,               # use all CPU cores
        )
        # RandomizedSearchCV wraps the base model with cross-validation
        # cv=3: 3-fold cross-validation for each parameter combination
        # scoring="f1": optimize for F1 score (better than accuracy for imbalanced
data)
        # n_iter=30: test 30 random parameter combinations
        search = RandomizedSearchCV(
            estimator=base_xgb,
            param_distributions=param_dist,
            n_iter=30,
            scoring="f1",
            cv=3,
            random_state=42,
            n_jobs=-1,
            verbose=0,
        )

        # Fit the search on the SMOTE-balanced training data
        search.fit(self.X_train, self.y_train)
        # The best estimator is the XGBoost model with the best hyperparameters
        self.xgb_model = search.best_estimator_

        print(f"[ModelTrainer] Best XGBoost params: {search.best_params_}")
        print(f"[ModelTrainer] Best CV F1 score: {search.best_score_:.4f}")

    def _ensemble_predict(self, X: np.ndarray) -> np.ndarray:

        # DNN: predict() returns shape (n, 1), flatten to (n,)
        dnn_probs = self.dnn_model.predict(X, verbose=0).ravel()
        # XGBoost: predict_proba() returns shape (n, 2), take column 1
(probability of hit)
        xgb_probs = self.xgb_model.predict_proba(X)[:, 1]
        # Simple average: weight each model equally
        ensemble_probs = (dnn_probs + xgb_probs) / 2.0
        return ensemble_probs

    def _find_best_threshold(self):
        # Get ensemble probabilities on the validation set
        val_probs = self._ensemble_predict(self.X_val)

        best_threshold = 0.5
        best_f1        = 0.0

        # Try 80 different thresholds between 0.10 and 0.90
```

```
        for t in np.arange(0.10, 0.91, 0.01):
            # Convert probabilities to 0/1 predictions using this threshold
            y_pred_t = (val_probs >= t).astype(int)

            # Compute precision and recall to get F1
            p = precision_score(self.y_val, y_pred_t, zero_division=0)
            r = recall_score(self.y_val, y_pred_t, zero_division=0)

            # F1 = harmonic mean of precision and recall
            if p + r > 0:
                f1 = 2 * p * r / (p + r)
                if f1 > best_f1:
                    best_f1        = f1
                    best_threshold = float(t)

        self.threshold = best_threshold
        print(f"[ModelTrainer] Optimal threshold: {self.threshold:.2f}  "
              f"(validation F1 = {best_f1:.4f})")

    def _evaluate(self) -> dict:

        # Get final predictions on the test set
        test_probs = self._ensemble_predict(self.X_test)
        y_pred     = (test_probs >= self.threshold).astype(int)

        # Compute all metrics
        acc  = accuracy_score(self.y_test, y_pred)
        prec = precision_score(self.y_test, y_pred, zero_division=0)
        rec  = recall_score(self.y_test, y_pred, zero_division=0)
        f1   = f1_score(self.y_test, y_pred, zero_division=0)

        # AUC: area under the ROC curve -- threshold-independent measure of
ranking quality
        # 1.0 = perfect, 0.5 = random guessing
        auc  = roc_auc_score(self.y_test, test_probs)

        # Print results table
        print("\n" + "=" * 55)
        print("  ENSEMBLE TEST SET RESULTS  (DNN + XGBoost)")
        print("=" * 55)
        print(f"  Threshold : {self.threshold:.2f}  (optimized on val set)")
        print(f"  Accuracy  : {acc*100:.2f}%")
        print(f"  Precision : {prec*100:.2f}%")
        print(f"  Recall    : {rec*100:.2f}%")
        print(f"  F1 Score  : {f1*100:.2f}%")
        print(f"  AUC-ROC   : {auc*100:.2f}%")
        print("=" * 55)
        print()
        print(classification_report(self.y_test, y_pred,
                                    target_names=["No Hit (0)", "Hit (1)"]))

        return {
            "accuracy":  acc,
            "precision": prec,
            "recall":    rec,
            "f1":        f1,
            "auc":       auc,
        }

    def _save_report(self, results: dict):
        path = os.path.join(self.report_dir, "model_report.txt")
        # Build the report as a list of strings, then join with newlines
        lines = [
            "=" * 55,
            "  MODEL REPORT: DNN + XGBoost Ensemble",
            "=" * 55,
            "",
            "── DNN Architecture ──────────────────────────────────",
```

```
            "  Input(12)",
            "  -> Dense(128, ReLU) + BatchNorm + Dropout(0.4)",
            "  -> Dense(64,  ReLU) + BatchNorm + Dropout(0.3)",
            "  -> Dense(32,  ReLU) + Dropout(0.2)",
            "  -> Dense(1,   Sigmoid)",
            "",
            "── XGBoost ──────────────────────────────────────────",
            f"  Best params: {self.xgb_model.get_params()}",
            "",
            "── Improvements Applied ─────────────────────────────",
            "  1. fillna(-1) + Has_Critic_Score / Has_User_Score flags",
            "  2. Target encoding: Publisher, Genre, Platform",
            "  3. SMOTE oversampling on train set (12% -> 50% hit rate)",
            "  4. XGBoost tuned with RandomizedSearchCV (30 iters, cv=3)",
            "  5. Threshold optimized on val set for max F1",
            "  6. Ensemble: DNN + XGBoost probability average",
            "",
            "── Test Set Metrics ─────────────────────────────────",
            f"  Threshold : {self.threshold:.2f}",
            f"  Accuracy  : {results['accuracy']*100:.2f}%",
            f"  Precision : {results['precision']*100:.2f}%",
            f"  Recall    : {results['recall']*100:.2f}%",
            f"  F1 Score  : {results['f1']*100:.2f}%",
            f"  AUC-ROC   : {results['auc']*100:.2f}%",
        ]
        with open(path, "w", encoding="utf-8") as f:
            f.write("\n".join(lines))
        print(f"[ModelTrainer] Report saved -> {path}")
```

## EK-6: requirements.txt

```
pandas>=1.5.0
numpy>=1.23.0
matplotlib>=3.6.0
seaborn>=0.12.0
scikit-learn>=1.2.0
imbalanced-learn>=0.10.0
tensorflow>=2.12.0
xgboost>=1.7.0
```

## EK-7: Proje Klasör Yapısı

```
video_game_hit_predictor/
├── data/
│   └── Video_Games_Sales_as_at_22_Dec_2016.csv   ← Ham veri seti
├── src/
│   ├── main.py            ← Pipeline giriş noktası
│   ├── data_loader.py     ← DataLoader sınıfı
│   ├── analyzer.py        ← DataAnalyzer sınıfı
│   ├── visualizer.py      ← Visualizer sınıfı
│   └── model_trainer.py   ← ModelTrainer sınıfı
├── outputs/
│   ├── graphs/
│   │   ├── 01_top10_games.png
│   │   ├── 02_genre_avg_sales.png
│   │   ├── 03_platform_sales.png
│   │   ├── 04_year_trend.png
│   │   ├── 05_regional_comparison.png
│   │   ├── 06_critic_vs_sales.png
│   │   ├── 07_publisher_power.png
│   │   ├── 08_confusion_matrix.png
│   │   └── 09_model_comparison.png
│   └── reports/
│       ├── analysis_report.txt
│       └── model_report.txt
└── requirements.txt
```